ДИСЦИПЛИНА «Разработка программных средств поиска и анализа киберугроз»

ИНСТИТУТ «Институт кибербезопасности и цифровых технологий»

КАФЕДРА КБ-2 «Информационно-аналитические системы кибербезопасности»

ВИД УЧЕБНОГО МАТЕРИАЛА Лекция

ПРЕПОДАВАТЕЛЬ Новиков Евгений Иванович

СЕМЕСТР 9 семестр 2026-2027 учебный год

ЛЕКЦИЯ 5. МАСШТАБИРОВАНИЕ И ПРЕОБРАЗОВАНИЕ ПРИЗНАКОВ В КИБЕРБЕЗОПАСНОСТИ

План лекции:

1. Масштабирование признаков.
2. Нелинейные преобразования признаков.
3. Метрики качества регрессионных моделей.

In [1]:
```
import warnings
warnings.filterwarnings('ignore')
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
import random
%config InlineBackend.figure_format = 'retina'
import scipy.stats as sts
pd.set_option('display.max_columns', None)
```

Масштабирование и преобразование признаков в кибербезопасности

1. Масштабирование признаков

На предыдущих занятиях мы изучили технологии очистки данных от ошибок и дубликатов, импутации пропущенных значений. После их применения мы получаем осмысленные, полные и готовые к анализу данные. Однако, если использовать эти данные для обучения модели, то в большинстве случае ее качество будет невысоким.

Понятие и цели масштабирования.

Предположим у нас есть данные: 443 — номер порта HTTPS и 0.003 — длительность TCP-сессии в секундах.

Для модели обнаружения DDoS-атак эти 3 миллисекунды могут быть критически важным сигналом: атакующий открывает тысячи соединений, каждое из которых живет доли секунды. А порт 443 — это просто константа, которая одинакова для всего легитимного HTTPS-трафика.

Но алгоритм машинного обучения этого не знает. Он видит числа. И для него 443 — это число, которое в 147000 раз больше, чем 0.003. Если мы используем алгоритм, который опирается на расстояния между точками (KNN, SVM, K-Means) или на градиентный спуск (нейросети, логистическая регрессия), признак «порт» практически полностью "задавит" признак "«длительность". Модель будет кластеризовать трафик по номерам портов, игнорируя все временные аномалии и она пропустит атаку.

Чтобы избежать подобных ситуаций и нужно масштабирование признаков.

Масштабирование признаков (feature scaling) — это процедура преобразования значений признаков к единому диапазону или распределению. Цель — устранить

различия в масштабах измерений между признаками, чтобы они оказывали сопоставимое влияние на модель.

Цели масштабирования:

1. Улучшение работы алгоритмов машинного обучения, основанных на расстояниях.

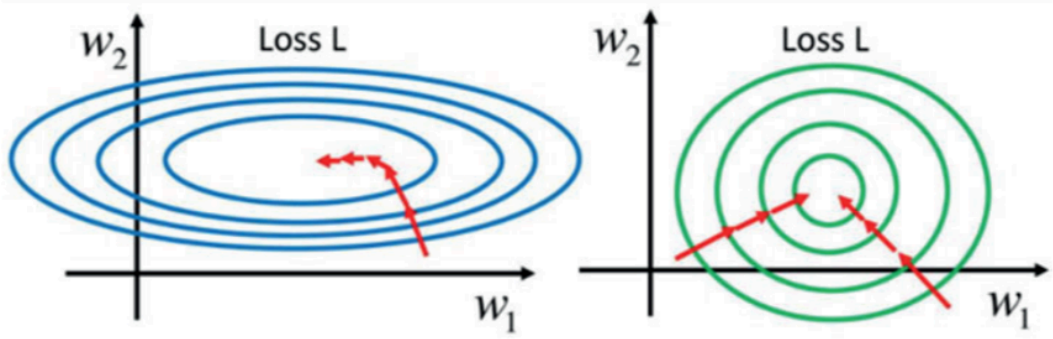
Существует класс метрических методов машинного обучения основаных на измерении сходства или различия между экземплярами данных: K-ближайших соседей (KNN), кластеризация (K-Means), опорных векторов (SVM). Результат их работы зависит от метрики расстояния (например, Евклидово).

Например, при поиске сходства наблюдений сетевого трафика признак "объём данных" может измеряться в диапазоне от 0 до 100000, а количество портов — в диапазоне от 0 до 10. При использовании метрических методов признак с большим диапазоном будет доминировать в расчётах.

2. Ускорение сходимости алгоритмов оптимизации.

Многие методы машинного обучения используют градиентный спуск (линейная/логистическая регрессия, градиентный бустинг, нейронные сети). Градиентный спуск ищет минимум функции потерь, двигаясь по поверхности «оврага». Если признаки в разных масштабах, поверхность потерь становится вытянутой, как длинный каньон, и градиентный спуск «зигзагами» ползет к минимуму, сходясь в десятки раз медленнее. А с масштабированием поверхность становится округлой, и спуск идет прямо к цели, т. е. алгоритмы сходятся быстрее.

На рисунке показано, как будет осуществлен градиентный спуск до масштабирования (слева) и после - (справа).



3. Повышение устойчивости регуляризации.

В моделях с L1/L2-регуляризацией (например, Ridge, Lasso) накладывается штраф на большие веса модели. Если признак x_1 имеет масштаб 1000, а x_2 - 0.01, то модель будет вынуждена присвоить признаку x_1 очень маленький вес, а x_2 - очень большой, просто чтобы компенсировать разницу масштабов. Регуляризация будет несправедливо «наказывать» признак x_2, хотя он может быть самым информативным.

4. Улучшение интерпретации коэффициентов.

После масштабирования веса модели становятся сопоставимыми: можно сравнивать важность признаков по величине коэффициентов.

Есть класс алгоритмов, которые **НЕ нуждаются** в масштабировании: Decision Trees, Random Forest, XGBoost, LightGBM, CatBoost. Это алгоритмы на основе деревьев решений. Масштабирование для них — это лишняя вычислительная нагрузка без прироста качества.

Рассмотрим технологии масштабирования признаков на следующем наборе данных.

In [3]:
```
# @title
np.random.seed(7)
n = 500

raw = np.random.normal(loc=300, scale=80, size=1000)
response_time = raw[raw > 0][:500]

data = {
    'dst_port': np.random.choice([22, 53, 80, 443, 8080, 3389, 445], size=n),
    'duration': np.random.exponential(scale=2.0, size=n),
    'bytes_sent': np.random.lognormal(mean=8, sigma=2.5, size=n),
    'n_packets': np.random.poisson(lam=30, size=n),
    'response_time': response_time,
    'entropy': np.random.uniform(0, 8, size=n),
}

df = pd.DataFrame(data)

outlier_idx = np.random.choice(n, size=10, replace=False)
df.loc[outlier_idx, 'bytes_sent'] = df.loc[outlier_idx, 'bytes_sent'] * 100
df.loc[outlier_idx, 'n_packets'] = df.loc[outlier_idx, 'n_packets'] * 50

df['severity_score'] = (
    np.log1p(df['bytes_sent']) * 3 +
    (1.0 / (df['duration'] + 0.1)) * 10 +
    df['entropy'] * 2 +
    np.random.normal(0, 5, size=n)
)

# Добавляем несколько явных выбросов (DDoS-атаки)
outlier_idx = np.random.choice(n, size=10, replace=False)
df.loc[outlier_idx, 'bytes_sent'] = df.loc[outlier_idx, 'bytes_sent'] * 100
df.loc[outlier_idx, 'n_packets'] = df.loc[outlier_idx, 'n_packets'] * 50

df['severity_score'] = df['severity_score'].clip(0, 100)
conditions = [
    (df['bytes_sent'] > 50000) & (df['n_packets'] > 200),
    (df['dst_port'] == 22) & (df['duration'] < 0.5) & (df['n_packets'] > 50),
    (df['bytes_sent'] > 10000) & (df['entropy'] > 6.5),
    (df['n_packets'] > 100) & (df['duration'] < 1.0),
]
choices = ['ddos', 'bruteforce', 'exfiltration', 'port_scan']
df['attack_type'] = np.select(conditions, choices, default='benign')
```

```
df.head()
```

Out[3]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523
2	445	0.219653	1905.247828	33	302.625613	0.526045	57.304126
3	53	3.228192	21542.663165	38	332.601303	1.379786	41.529896
4	80	2.398801	81330.193172	35	236.886158	1.966413	37.332529

Признаки:

- `dst_port` : порт назначения;
- `duration` : длительность сессии (сек);
- `bytes_sent` : количество переданных байт;
- `n_packets` : количество пакетов;
- `response_time` : время отклика веб-сервера (мс);
- `entropy` : энтропия payload;
- `severity_score` : оценка критичности инцидента;
- `attack_type` : тип атаки.

Посмотрим на разброс значений признаков.

In [4]:
```
df.describe().loc[['min', 'max']].round(2)
```

Out[4]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_sco
min	22.0	0.00	1.110000e+00	16.0	53.40	0.00	8.
max	8080.0	12.44	1.915556e+08	1950.0	528.89	7.98	100.

При использовании линейных моделей и метрических алгоритмов признак `bytes_sent` будет определять 99.9% этого расстояния. Все остальные признаки будут «невидимы» для модели.

Если строить регрессионную модель предсказания критичности инцидента, то она будет предсказывать целевой признак `severity_score` только по объему трафика, игнорируя энтропию и длительность.

В случае построения модели классификации типа атаки (`attack_type`) она не сможет отличить `bruteforce` от `port_scan` , потому что оба типа имеют короткие сессии, но значительно различаются по `n_packets` и `dst_port` .

Методы масштабирования

По сути все методы масштабирования основаны на том, что исходный набор значений первоначально смещается, а потом масштабируется:

$$x_i^* = \frac{x_i - sm}{int},$$

- x_i^* - i-е значение масштабированного признака;
- x_i - i-е значение исходного признака;
- sm - величина смещения значений признака;
- int - величина интервала, который будет преобразован к "единице".

Встречаются различные названия методов маштабирования. Здесь и в дальнейшем будем использовать для обозначения методов названия соответствующих классов библиотеки `scikit-learn`.

StandardScaler

StandardScaler - стандартизация методом z-оценки (z-score). Это статистический метод преобразования данных, который позволяет выразить каждое значение признака в терминах стандартных отклонений от среднего значения выборки. Метод преобразует данные так, что новое распределение имеет среднее значение 0 и стандартное отклонение 1, что упрощает сравнение данных с разными масштабами или единицами измерения.

Преобразование каждого значения признака выполняется по формуле z-score:

$$z_i = \frac{x_i - \mu}{\sigma},$$

где:

- μ - математическое ожидание признака;
- σ - стандартное отклонение признака.

После преобразования:

- $\mu = 0$;
- $\sigma = 1$;
- диапазон значений: $(-\infty, +\infty)$ — не ограничен.

Рассмотрим пример стандартизации независимых признаков.

Для стандартизации используется класс `StandardScaler` модуля `preprocessing` библиотеки `sklearn`.

In [5]:
```
from sklearn.preprocessing import StandardScaler
```

При вызове метода `fit` алгоритм стандартизации рассчитывает среднее арифметическое и стандартное отклонение для каждого признака. Их можно посмотреть через соответствующие атрибуты.

Метод `transform` использует рассчитанные значения среднего и стандартного отклонения для стандартизации данных и возвращает массив `numpy` с преобразованными значениями.

Метод `fit_transform` объединяет вычисление числовых характеристик и преобразование и сразу вычисляет статистические показатели и применяет их для масштабирования данных.

In [6]:
```
scaler_std = StandardScaler()
df_std = pd.DataFrame(
    scaler_std.fit_transform(df.drop(columns=['severity_score', 'attack_type'])),
    columns=df.drop(columns=['severity_score', 'attack_type']).columns)

df_std.head()
```

Out[6]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy
0	0.603224	-0.281353	-0.057955	-0.205105	1.797293	-1.683015
1	-0.598642	-0.297687	-0.057784	-0.235739	-0.410521	0.775822
2	-0.466070	-0.919833	-0.057797	-0.191491	0.100114	-1.528856
3	-0.608448	0.651165	-0.055510	-0.174472	0.483732	-1.163335
4	-0.598642	0.218073	-0.048546	-0.184683	-0.741197	-0.912176

Числовые характеристики стандартизованных признаков.

In [7]:
```
df_std.describe().loc[['min', 'max', 'mean', 'std']].T.round(4)
```

Out[7]:

	min	max	mean	std
dst_port	-0.6197	2.3070	-0.0	1.001
duration	-1.0330	5.4598	-0.0	1.001
bytes_sent	-0.0580	22.2531	-0.0	1.001
n_packets	-0.2494	6.3334	0.0	1.001
response_time	-3.0894	2.9957	-0.0	1.001
entropy	-1.7521	1.6625	-0.0	1.001

Условия применения StandardScaler.

1. Нормальное распределение признаков. Z-оценка наиболее эффективна и интерпретируема при условии, что исходные данные распределены нормально. Потому что для нормального распределения работают правила "сигм".

2. Отсутствие или незначительность выбросов. Поскольку Z-оценка использует среднее и стандартное отклонение, которые чувствительны к экстремальным значениям, наличие выбросов может исказить результаты.

3. Непрерывные данные. Метод применим только к непрерывным числовым переменным. Категориальные или порядковые данные не подходят для Z-оценки.

Почему "правила сигм" не работают?

StandardScaler математически корректен и наиболее эффективен, когда исходные данные имеют нормальное (гауссово) распределение. Потому что для нормального распределения работают правила "сигм":

$\approx 68\%$ данных попадают в диапазон $[\mu - \sigma, \mu + \sigma]$, то есть после стандартизац

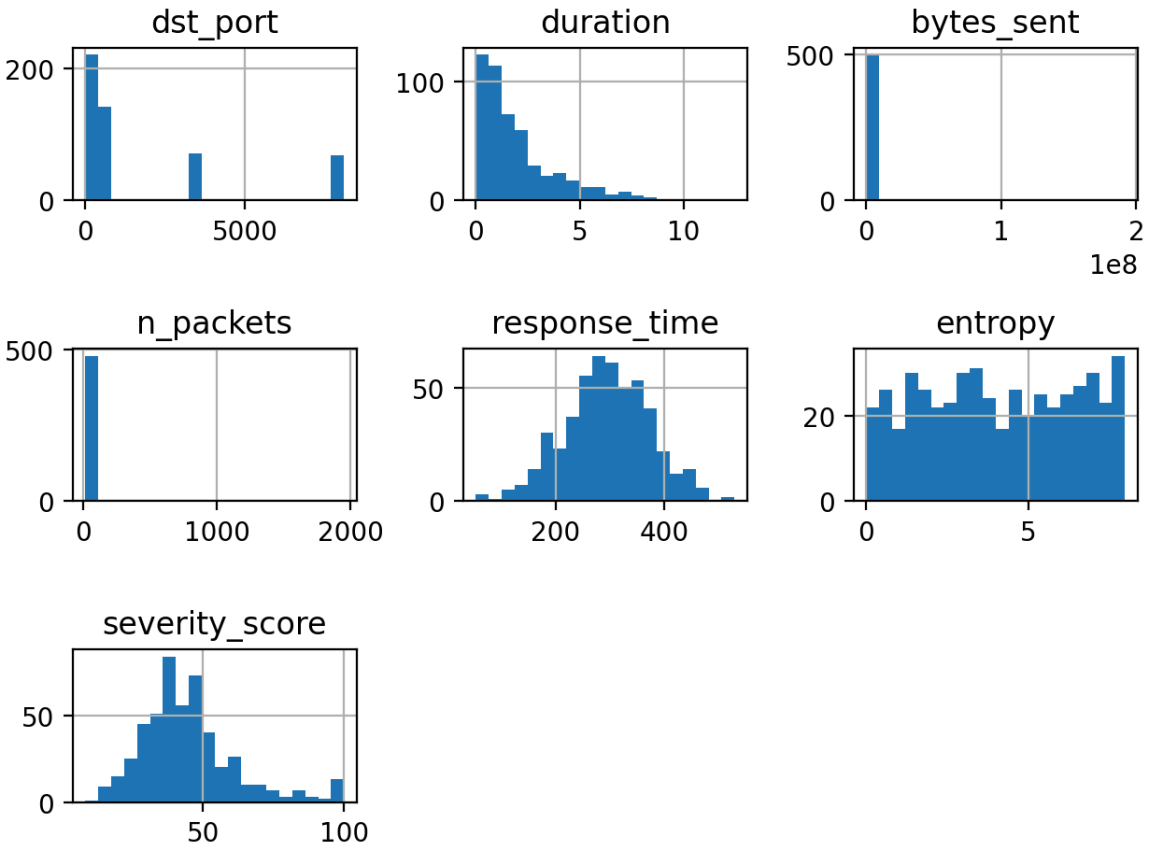
$\approx 95\%$ данных попадают в диапазон $[\mu - 2\sigma, \mu + 2\sigma]$, то есть после стандартиз

$\approx 99\%$ данных попадают в диапазон $[\mu - 3\sigma, \mu + 3\sigma]$, то есть после стандартиз

Это означает, что после стандартизации нормального распределения почти все значения оказываются в компактном диапазоне $[-3, 3]$.

Визуальный анализ законов распределения признаков.

In [10]:
```
df.hist(bins = 20)
plt.tight_layout();
```



Все независимые признаки, кроме признака `response_time`, имеют закон распределения отличные от нормального.

Для признака `response_time` действительно выполняется правило "3 сигм".

In [11]:
```
print(f" В диапазоне [-1, 1]: {(np.abs(df_std['response_time']) <= 1).mean():.1%
print(f" В диапазоне [-2, 2]: {(np.abs(df_std['response_time']) <= 2).mean():.1%
print(f" В диапазоне [-3, 3]: {(np.abs(df_std['response_time']) <= 3).mean():.1%
print(f" За пределами [-3, 3]: {(np.abs(df_std['response_time']) > 3).mean():.1%
```

```
В диапазоне [-1, 1]: 67.4%
В диапазоне [-2, 2]: 95.6%
В диапазоне [-3, 3]: 99.8%
За пределами [-3, 3]: 0.2%
```

Рассмотрим для примера признак `duration`, который имеет экспоненциальный закон распределения.

Для экспоненциального закона справедливо:

In [12]:
```
print(f" В диапазоне [-1, 1]: {(np.abs(df_std['duration']) <= 1).mean():.1%}")
print(f" В диапазоне [-2, 2]: {(np.abs(df_std['duration']) <= 2).mean():.1%}")
print(f" В диапазоне [-3, 3]: {(np.abs(df_std['duration']) <= 3).mean():.1%}")
print(f" За пределами [-3, 3]: {(np.abs(df_std['duration']) > 3).mean():.1%}")
```

```
В диапазоне [-1, 1]: 82.4%
В диапазоне [-2, 2]: 94.4%
В диапазоне [-3, 3]: 98.4%
За пределами [-3, 3]: 1.6%
```

В диапазон $[-1, 1]$ попадает на 15% больше данных, чем для нормального распределения (68%). А за пределами $[-3, 3]$ остается 1.6% данных — это в 8 раз больше, чем для нормального распределения (0.2%).

К чему это приводит?

Предположим для обнаружения аномалий в значениях признака `duration` используется правило "3 сигм" — «если значение отклоняется от среднего более чем на 3 стандартных отклонения ($|Z| > 3$), то оно считается аномалией и инициируется сигнал тревоги».

Количество ложных срабатываний при использовании этого правила зависит от вида закона рапсределения признака.

В кибербезопасности False Positive (FP) — это ситуация, когда модель оценила событие как атака, а на самом деле это нормальный, легитимный трафик, просто немного необычный.

Например:

1. Администратор делает большой бэкап базы данных → `bytes_sent` резко вырос.
2. Пользователь скачивает большой файл → `duration` увеличился.
3. Сервер делает массовую рассылку → `n_packets` значительно увеличился.

Это не атаки. Но если эти значения попадают в «хвост» распределения, модель (или правило «3 сигм») может ошибочно пометить их как аномалии. Это и есть ложные срабатывания.

Предположим, что собраны данные о 10000 сетевых сессий за час.

При этом реальных атак: 10 (это 0.1% от трафика). Нормального трафика: 9990 (это 99.9% от трафика).

Применяем правило "правило 3 сигм".

Сценарий 1. Закон распределения признака - нормальный.

Например, как для признака `response_time`.

По «правилу 3 сигм» за пределы диапазона $[-3, 3]$ выходит 0.2% всех данных. Т. е. 0.2% от 10000 = 20 сессий попали под признак атаки.

Из этих 20 сессий:

- 10 сессий — это реальные атаки (True Positive). Они выявлены моделью;
- 10 сессий — это нормальный трафик, который просто случайно оказался в «хвосте» нормального распределения (False Positive).

Таким образом, количество ложных срабатываний равно 10. И аналитик SOC проанализирует их за короткое время, т. е. - это допустимый уровень ложных срабатываний.

Сценарий 2: Закон распределения признака экспоненциальный.

Например, как для признака `duration`.

Применяем то же самое правило «всё, что за пределами $[-3, 3]$ — это атака». Но для экспоненциального распределения за эти пределы выходят уже 1.6% данных.

Тогда 1.6% от 10000 = 160 сессий попали под признак атаки.

Из этих 160 сессий:

- 10 сессий — это реальные атаки (True Positive). Они выявлены моделью;
- 150 сессий — это нормальный трафик, который для экспоненциального распределения является «типичным хвостом», но используемое «правило 3 сигм» посчитало его аномалией (False Positive).

Таким образом, количество ложных срабатываний равно 150.

Количество ложных срабатываний выросло почти в 9 раз. При этом количество найденных реальных атак (True Positive) осталось прежним — 10. Мы не стали лучше выявлять атаки, но мы в 10 раз увеличили нагрузку на аналитиков SOC, которые теперь вынуждены разбирать 150 ложных тревог вместо 10.

Если перенести это на реальные объемы (миллионы событий в день), то вместо сотни ложных срабатываний в день аналитики будут получать сотни тысяч. Это приведет к тому, что они просто перестанут реагировать на предупреждения, пропустив при этом реальную атаку.

Рассмотрим еще один пример некорректного использования стандартизации признаков.

Признак `entropy` имеет равномерное распределение.

После стандартизаци получаем:

In [14]:
```
print("\nРавномерное распределение (entropy):")
print(f" В диапазоне [-1, 1]: {(np.abs(df_std['entropy']) <= 1).mean():.1%}")
print(f" В диапазоне [-2, 2]: {(np.abs(df_std['entropy']) <= 2).mean():.1%}")
print(f" В диапазоне [-3, 3]: {(np.abs(df_std['entropy']) <= 3).mean():.1%}")
print(f" За пределами [-3, 3]: {(np.abs(df_std['entropy']) > 3).mean():.1%}")
```

```
Равномерное распределение (entropy):
 В диапазоне [-1, 1]: 57.0%
 В диапазоне [-2, 2]: 100.0%
 В диапазоне [-3, 3]: 100.0%
 За пределами [-3, 3]: 0.0%
```

В диапазон $[-1, 1]$ попадает только 57.4% данных — меньше, чем для нормального распределения.

За пределами $[-3, 3]$ — 0% данных.

Если использовать "правило 3 сигм" для обнаружения аномалий в признаке `entropy`, то сигнал об атаке никогда не будет получен. Но это не значит, что аномалий нет. Просто стандартизация сжала все значения признака в допустимый диапазон.

Таким образом, применение стандартизации к признакам имеющим отличные от нормального распределения приводит к неправильной настройке порогов сигналов атаки.

Если настроить порог сигнала на `Z > 3` (ожидая 0.2% ложных срабатываний), то:

- для нормального распределения: будет получено 0.2% ложных срабатываний ✓
- для экспоненциального: будет получено 1.6% ложных срабатываний (в 8 раз больше!) ✗
- для логнормального: будет получено 0.7% ложных срабатываний (в 3.5 раза больше) ✗
- для равномерного: никогда не будет получен сигнал атаки (0%) ✗

Почему теряется интерпретируемость?

Интерпретируемость Z-score — это когда число имеет единый, универсальный смысл, понятный любому аналитику SOC.

Когда данные имеют нормальное распределение, Z-score становится таким «универсальным языком».

Для нормального распределения Z-score имеет четкую вероятностную интерпретацию:

$$Z = 2 \Longrightarrow P(X > x) = P(Z > 2) = 1 - \Phi(2) \approx 2.28\%.$$

Если аналитик SOC видит сигнал: «Z-score = 2», ему не нужно знать, измеряется трафик в байтах, килобайтах или мегабайтах. Ему не нужно смотреть на гистограмму. Он мгновенно понимает, что событие происходит примерно 1 раз из 43. Это необычно, но не критично. Детальное рассмотрение этого события можно отложить.

Если аналитик SOC, видит сообщение: «Z-score = 3», то сразу понимает, что это значение встречается реже, чем 1 раз на 1000 сессий. Это точно аномалия и ее анализ нужно проводить оперативно.

In [16]:
```
df['duration'].std()
```

Out[16]:
```
1.9169678601619689
```

Посмотрим, что происходит для ненормальных распределений.

Пример. Признак `duration` имеет экспоненциальное распределение.

После его стандартизации получили одно из значений $Z = 2$. Найдём, какое исходное значение признака x даст $Z = 2$ ($\mu \approx 2$, $\sigma \approx 2$):

$$Z = \frac{x - \mu}{\sigma} = 2 \Rightarrow x = \mu + 2\sigma = 6.$$

Надём реальный процентиль для $x = 6$ в экспоненциальном распределении ($\lambda = 1/\mu = 0.5$):

$$P(X \leq 6) = 1 - e^{-\lambda x} = 1 - e^{-0.5 \cdot 6} \approx 0.9502 = 95.0\%.$$

Проблема 1: Невозможность сравнения признаков.

В нормальном распределении $Z = 2$ соответствует 97.7-му процентилю, а в экспоненциальном (даже после такой же формулы стандартизации) $Z = 2$ даёт лишь 95-й процентиль. То есть «типичность» одного и того же значения Z разная.

Проблема 2: Неправильная интерпретация аномалий.

Если строить детектор аномалий на основе правила "3 сигм", то будет систематически занижаться порог аномальности для экспоненциальных признаков: то, что выглядит как «2 сигмы», на самом деле — лишь 95 % покрытие, а не 97.7 %, и это будет давать гораздо больше ложноположительных срабатываний (или, наоборот, пропускать реальные аномалии, если порог завышен).

Рассмотрим влияние выбросов на результаты стандартизации.

Выбросы в кибербезопасности — это не просто "шум". Это часто целевые атаки: DDoS с гигантским трафиком, brute-force с тысячами попыток.

Посмотрим, как выбросы влияют на параметры μ и σ математически.

Предположим у нас есть 12 значений признака `bytes_sent`: 11 нормальных сессий и 1 - DDoS-атака: $x_i = 10\ 000\ 000$ байт.

Вычислим математическое ожидание и стандартное отклонение для данных без выбросов (только 11 нормальных сессий) и для данных с выбросом.

In [17]:
```
bytes_sent_out = [1000,1200,1100,1300,1050,1150,1250,1400,1180,1220,1350,1000000
print('Без выбросов:')
print(f"μ={np.mean(bytes_sent_out[:-1])}")
print(f"σ={np.std(bytes_sent_out[:-1])}")
print()
print('С выбросами:')
print(f"μ={np.mean(bytes_sent_out)}")
print(f"σ={np.std(bytes_sent_out)}")
```

```
Без выбросов:
μ=1200.0
σ=117.08582710597773

С выбросами:
μ=834433.3333333334
σ=2763522.331757466
```

Среднее μ увеличилось почти в 700 раз, а стандартное отклонение σ - в 23 600.

Стандартизуем нормальную сессию с `bytes_sent = 1200` для данных без выброса и с выбросом.

In [18]:
```
z = (1200 - np.mean(bytes_sent_out[:-1])) / np.std(bytes_sent_out[:-1])
print(f'Без выбросов: z={z}')
z_outlier = (1200 - np.mean(bytes_sent_out)) / np.std(bytes_sent_out)
print(f'С выбросами: z={z_outlier}')
```

```
Без выбросов: z=0.0
С выбросами: z=-0.30151134432969734
```

Нормальная сессия, которая должна была иметь $z = 0$, теперь выглядит как "ниже среднего", хотя на самом деле это типичное значение.

Стандартизуем все значения признака для данных с выбросом.

In [19]:
```
scaler_std = StandardScaler()
Z = scaler_std.fit_transform(np.array(bytes_sent_out).reshape(-1, 1))
Z
```

Out[19]:
```
array([[-0.30158372],
       [-0.30151134],
       [-0.30154753],
       [-0.30147516],
       [-0.30156562],
       [-0.30152944],
       [-0.30149325],
       [-0.30143897],
       [-0.30151858],
       [-0.30150411],
       [-0.30145707],
       [ 3.31662479]])
```

Стандартизация сжимает диапазон «нормальных» данных. Большинство значений после масштабирования находятся в сжатом узком диапазоне. Теряется информация об относительных расстояниях между наблюдениями, не являющимися выбросами. Это затрудняет обнаружение небольших отклонений от нормы, которые также могут быть подозрительными.

Для аномального значения признака получено большое, но не "экстремальное" значение.

Стандартизация "размазывает" выбросы. Они не исчезают, но становятся менее заметными на фоне "растянутого" масштаба. Модель может не заметить аномалию, потому что `z-score = 3.31` — это не так уж много для алгоритма, который ожидает значения в диапазоне $[-10, 10]$.

Стандартизацию можно применять к признакам с выбросами в следующих случаях:

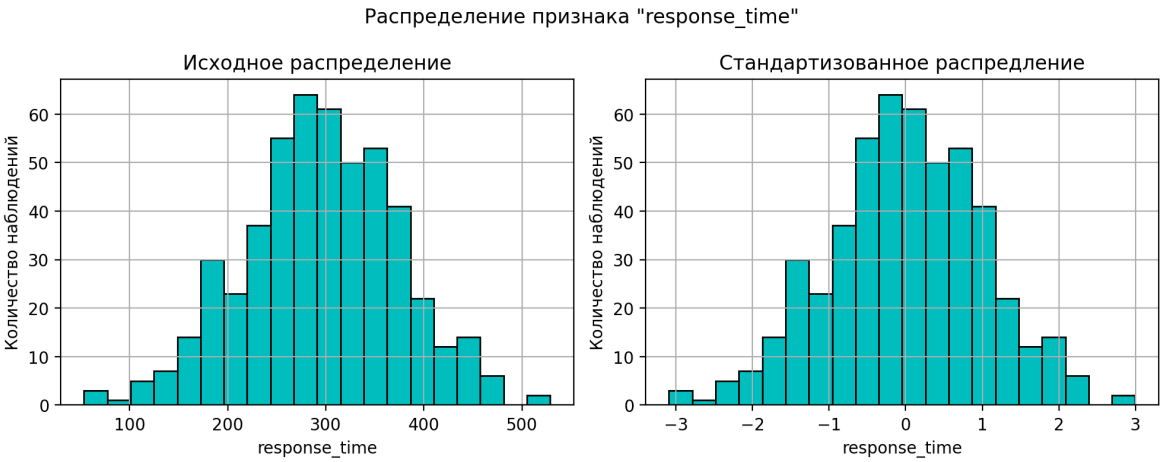
1. Если выбросов очень мало (< 1% данных) и они не экстремальны (в пределах 10-20 σ).
2. Если предварительно удалить выбросы.
3. Если алгоритм устойчив к выбросам (например, Random Forest не чувствителен к масштабу, поэтому ему всё равно, каким способом масштабировать данные).

Стандартизация признаков не меняет форму заспределения.

Сравним законы распределения признака `response_time` до и после стандартизации.

In [20]:
```
plt.figure(figsize = (10, 4))
plt.suptitle('Распределение признака "response_time"')
plt.subplot(1, 2, 1)
plt.hist(df['response_time'], bins = 20,
         color = 'c', edgecolor = 'black')
plt.title('Исходное распределение')
plt.xlabel('response_time')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.subplot(1, 2, 2)
plt.hist(df_std['response_time'], bins = 20,
         color = 'c', edgecolor = 'black')
plt.title('Стандартизованное распредление')
plt.xlabel('response_time')
```

```
plt.ylabel('Количество наблюдений')
plt.grid()
plt.tight_layout(pad = 1)
```



StandardScaler, как и любой метод масштабирования, который выполняет операцию ЦЕНТРИРОВАНИЯ (вычитает среднее или медиану), уничтожает разреженность для любого столбца, у которого центр (среднее или медиана) НЕ равен нулю.

Если есть разреженная матрица (много нулей), например, после TF-IDF векторизации логов, StandardScaler уничтожит разреженность.

In [21]:
```
import numpy as np
from scipy.sparse import csr_matrix
# Создаем разреженную матрицу (имитация TF-IDF) - 70% нулей
data = np.array([[1, 0],
                 [0, 2],
                 [3, 0],
                 [0, 0],
                 [0, 0]])
sparse_data = csr_matrix(data)

print(f"Исходная разреженность: {(sparse_data == 0).mean():.1%}\n")

# StandardScaler не принимает sparse матрицы напрямую,
# если with_centering=True (по умолчанию), он выдаст ошибку или неявно сделает t
std_scaler = StandardScaler()
dense_std = std_scaler.fit_transform(sparse_data.toarray())
print("После StandardScaler:")
print(np.round(dense_std, 2))
print(f"Разреженность: {(dense_std == 0).mean():.1%}\n")
```

```
Исходная разреженность: 70.0%

После StandardScaler:
[[ 0.17 -0.5 ]
 [-0.69  2.  ]
 [ 1.89 -0.5 ]
 [-0.69 -0.5 ]
 [-0.69 -0.5 ]]
Разреженность: 0.0%
```

В большинстве случаев масштабируются только независимые признаки.

Масштабирование целевого признака может быть полезно в задачах регрессии с градиентным спуском.

Если используется нейросеть или линейная регрессия с градиентным спуском для предсказания вещественного целевого признака, то его масштабирование ускоряет сходимость.

Предположим решается задача прогнозирования оценки критичности инцидента (`severity_score`).

В качестве модели машинного обучения выбрана линейная регрессия:

$$\text{severity_score} = \sum_{i=1}^{m} b_i \cdot x_i.$$

Процесс обучения модели и формирование прогноза.

In [22]:
```
from sklearn.linear_model import LinearRegression
X = df.drop(columns=['severity_score', 'attack_type'])
y = df[['severity_score']]

# Стандартизация обучающих данных
scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_scaled = scaler_X.fit_transform(X)
y_scaled = scaler_y.fit_transform(y)

# Обучение модели на исходных данных
model = LinearRegression().fit(X, y)

# Обучение модели на стандартизированных данных
model_st = LinearRegression().fit(X_scaled, y_scaled)

# Прогноз "severity_score" для новых данных
X_test = np.array([[80, 3.17, 2012, 15, 402, 3]])
X_test_scaled = scaler_X.transform(X_test)
y_pred = model.predict(X_test) # в исходных данных
y_pred_scaled = model_st.predict(X_test_scaled) # в Z-оценках

print(f'Прогноз в исходных единицах {y_pred}')
print(f'Прогноз в стандартизованных единицах {y_pred_scaled}')
```

```
Прогноз в исходных единицах [[36.92121003]]
Прогноз в стандартизованных единицах [[-0.45452948]]
```

В случае стандартизации как независимых, так и целевого признаков необходимо производить обратное преобразование значений целевого признака.

Причины обратного преобразования признаков.

1. Интерпретация предсказаний моделей в реальных единицах измерения.

   Модель обучается на стандартизированных данных, но пользователю нужен прогноз в исходных единицах.

В нашем примере модель формирует прогноз критичности инцидента (`severity_score`) в безразмерных единицах. Чтобы SOC-аналитику использовать эту модель для определения инцидентов с прогнозируемой высокой степенью отклонения нужно вернуться к исходному масштабу.

2. Сравнение метрик качества моделей с бизнес-требованиями.

   Метрики качества моделей (например, разница между истинным значением степени отклонения и модельным) на стандартизированных данных безразмерны. Для оценки качества в реальных терминах (например, "ошибка не более 0,8") требуется обратное преобразование.

3. Передача данных в другие системы.

   Внешние сервисы и базы данных ожидают значения в исходном формате.

4. Формирование порогов принятия решений.

   Пороги срабатывания (например, "если критичность инцидента > 80, устанавливается наивысший приоритет реагирования») задаются в исходных единицах.

Для обратного преобразования признаков используется метод `inverse_transform()` модуля `preprocessing` библиотеки `sklearn` .

При этом в качестве преобразователя необходимо использовать преобразователь, который был создан для стандартизации целевого признака при обучении обучении модели.

В нашем примере для стандартизации целевого признака `severity_score` был создан преобразователь `scaler_y` .

In [25]:
```
scaler_y
```

Out[25]:
```
StandardScaler
StandardScaler()
```

Его необходимо использовать для преобразования прогнозного значения целевого признака в исходные единицы измерения.

In [24]:
```
scaler_y.inverse_transform(y_pred_scaled)
```

Out[24]:
```
array([[36.92121003]])
```

RobustScaler

StandardScaler работает идеально для нормальных распределений, но не пригоден при наличии выбросов и асимметрии. А в кибербезопасности выбросы — это

норма, а не исключение. DDoS-атаки, эксфильтрация данных, брутфорс — всё это экстремальные значения в признаках `bytes_sent`, `duration`, `n_packets`.

Поэтому нужен метод, который при наличии выбросов не позволяет им доминировать над масштабированием. Таким методом является RobustScaler.

Этот метод использует статистики, устойчивые к выбросам: медиану вместо среднего, интерквантильный размах (IQR) вместо стандартного отклонения:

$$r_i = \frac{x_i - Me(X)}{IQR(X)},$$

где:

- $Me(X)$ - медиана признака;
- где: $IQR(X) = Q_3 - Q_1$ - интерквантильный размах;
- Q_1 - первый квартиль (25-й процентиль) — значение, ниже которого находятся 25% данных;
- Q_3 - третий квартиль (75-й процентиль) — значение, ниже которого находятся 75% данных.

IQR измеряет разброс центральных 50% данных, игнорируя крайние 25% с каждой стороны. Это делает IQR устойчивым к выбросам в хвостах распределения.

Результат преобразования:

- $Me(X) = 0$;
- $IQR(X) = 1$;
- диапазон значений: $(-\infty, +\infty)$ — не ограничен.

Почему медиана и IQR устойчивы к выбросам?

Предположим есть значения признака `bytes_sent` (байт).

In [26]:
```
bytes_sent = [1000,1200,1100,1300,1050,1150,1250,1400,1180,1220,1350]
```

Вычислим среднее значение и медиану.

In [27]:
```
print(f'Среднее значение: {np.mean(bytes_sent)}')
print(f'Медиана: {np.median(bytes_sent)}')
```

```
Среднее значение: 1200.0
Медиана: 1200.0
```

Теперь добавим один DDoS-выброс: 10 000 000 (10 ГБ) и пересчитаем среднее арифметическое и медиану.

In [28]:
```
bytes_sent_out = [1000,1200,1100,1300,1050,1150,1250,1400,1180,1220,1350,1000000
print(f'Среднее значение: {np.mean(bytes_sent_out)}')
print(f'Медиана: {np.median(bytes_sent_out)}')
```

```
Среднее значение: 834433.3333333334
Медиана: 1210.0
```

Среднее увеличилось почти в 700 000 раз, а медиана практически не изменилась.

Теперь для этих же значений вычислим интерквантильный размах.

In [29]:
```
Q1 = np.quantile(bytes_sent, 0.25)
Q3 = np.quantile(bytes_sent, 0.75)
IQR = Q3 - Q1
print(f'Q1 без выбросов: {Q1}')
print(f'Q3 без выбросов: {Q3}')
print(f'Интерквантильный размах без выбросов: {IQR}')
print()
Q1_out = np.quantile(bytes_sent_out, 0.25)
Q3_out = np.quantile(bytes_sent_out, 0.75)
IQR_out = Q3_out - Q1_out
print(f'Q1 с выбросом: {Q1_out}')
print(f'Q3 с выбросом: {Q3_out}')
print(f'Интерквантильный размах с выбросом: {IQR_out}')
```

```
Q1 без выбросов: 1125.0
Q3 без выбросов: 1275.0
Интерквантильный размах без выбросов: 150.0

Q1 с выбросом: 1137.5
Q3 с выбросом: 1312.5
Интерквантильный размах с выбросом: 175.0
```

Метод RobustScaler игнорирует выбросы при вычислении параметров масштабирования. Медиана и межквартильный размах (IQR) рассчитываются по центральной части распределения, поэтому экстремальные значения практически не влияют на масштабирование.

Вернемся к примеру с DDoS-атаками и покажем, как RobustScaler решает проблемы, которые имеет StandardScaler.

In [30]:
```
bytes_sent_out
```

Out[30]:
```
[1000, 1200, 1100, 1300, 1050, 1150, 1250, 1400, 1180, 1220, 1350, 10000000]
```

Произведем масштабирование значений признака `bytes_sent` с использованием RobustScaler.

In [31]:
```
from sklearn.preprocessing import RobustScaler
scaler_r = RobustScaler()
R = scaler_r.fit_transform(np.array(bytes_sent_out).reshape(-1, 1))
R
```

Out[31]:
```
array([[-1.20000000e+00],
       [-5.71428571e-02],
       [-6.28571429e-01],
       [ 5.14285714e-01],
       [-9.14285714e-01],
       [-3.42857143e-01],
       [ 2.28571429e-01],
       [ 1.08571429e+00],
       [-1.71428571e-01],
       [ 5.71428571e-02],
       [ 8.00000000e-01],
       [ 5.71359429e+04]])
```

Решение RobustScaler: сохранение различий.

При наличии выбросов среднее "сдвигается" в их сторону, а медиана остаётся ближе к основной массе данных. Поэтому при использовании RobustScaler значения признака концентрируются вокруг нуля, а не «разбегаются» в отрицательную область как при использовании StandardScaler.

При использовании RobustScaler отражается реальный разброс основной массы данных - обычные значения признака сохраняют свою вариативность после преобразования.

Решение RobustScaler: явное выделение выбросов.

При использовании среднего для аномального значения признака 10 000 000 после стандартизации было получено $z = 3.31$.

При использовании Robust Scaling для выброса получаем значение ≈ 57 135, что четко отделяет его от нормальных сессий (которые лежат в диапазоне $[-1.2, 1.08]$). Модель однозначно определяет, что это аномалия.

Рассмотрим пример применения RobustScaler для реального признака `bytes_sent` и сравним его со StandardScaler.

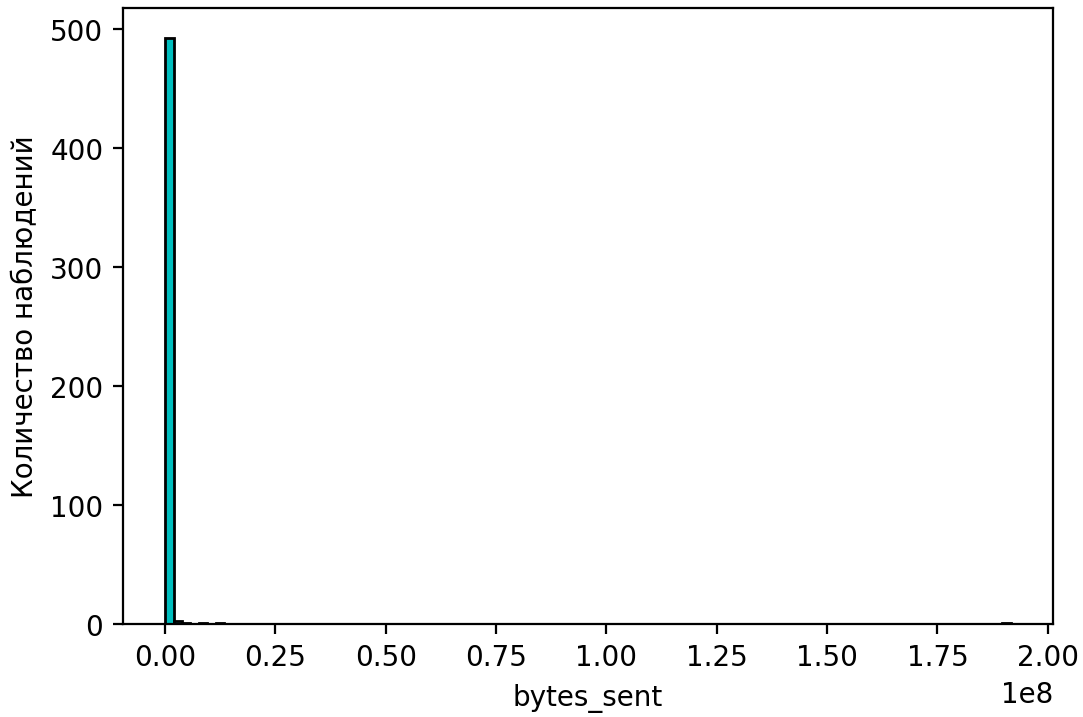
In [32]:
```
df.head(2)
```

Out[32]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523

Оценим закон распределения признака.

In [35]:
```
plt.figure(figsize = (6, 4))
plt.hist(df['bytes_sent'], bins = 100,
         color = 'c', edgecolor = 'black')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений');
```



Признак имеет логнормальное распределение с выбросами.

In [36]:
```
print(f" Медиана: {df['bytes_sent'].median():,.0f}")
print(f" IQR: {df['bytes_sent'].quantile(0.75) - df['bytes_sent'].quantile(0.25)
print()
print(f" Среднее: {df['bytes_sent'].mean():,.0f}")
print(f" Стандартное отклонение: {df['bytes_sent'].std():,.0f}")
```

```
Медиана: 2,641
IQR: 19,059

Среднее: 498,133
Стандартное отклонение: 8,594,268
```

Произведем масштабирование с использованием StandardScaler и RobustScaler.

In [37]:
```
std_scaler = StandardScaler()
df['bytes_std'] = std_scaler.fit_transform(df[['bytes_sent']])

robust_scaler = RobustScaler()
df['bytes_robust'] = robust_scaler.fit_transform(df[['bytes_sent']])

print("\nПосле StandardScaler:")
print()
print(f"  Медиана Z: {df['bytes_std'].median():.3f} - не 0! из-за асимметрии")
print(f"  Диапазон нормальных Z: [{df['bytes_std'].min():.3f}, {df['bytes_std'].
print(f"  Максимум Z: {df['bytes_std'].max():.3f} - выбросы не отделены от нормы

print("\nПосле RobustScaler:")
print()
print(f"  Медиана R: {df['bytes_robust'].median():.3f}")
print(f"  Диапазон нормальных R: [{df['bytes_robust'].min():.3f}, {df['bytes_rob
print(f"  Максимум R: {df['bytes_robust'].max():.3f} - выбросы четко отделены от
```

```
После StandardScaler:

  Медиана Z: -0.058 - не 0! из-за асимметрии
  Диапазон нормальных Z: [-0.058, -0.024] - нормальные сессии находятся в небольш
ом диапазон
  Максимум Z: 22.253 - выбросы не отделены от нормы

После RobustScaler:

  Медиана R: 0.000
  Диапазон нормальных R: [-0.139, 15.005] - нормальные сессии имеют значительную
вариацию
  Максимум R: 10050.484 - выбросы четко отделены от нормы
```

Таким образом, RobustScaler обладает следующими преимуществами перед StandardScaler:

- Устойчивость к выбросам. Медиана и IQR не зависят от экстремальных значений, поэтому RobustScaler не искажает масштаб данных при наличии аномалий.
- Сохранение структуры данных. Нормальные наблюдения остаются рядом друг с другом. Аномальные - остаются далеко от нормы. Модель может легко провести границу между ними.
- Работа с асимметричными распределениями. RobustScaler не предполагает нормальности данных и подходит для искажённых распределений.

Особенности RobustScaler.

Так же как и StandardScale не изменяет форму распределения.

In [38]:
```
plt.figure(figsize = (10, 4))
plt.suptitle('Распределение признака "bytes_sent"')
plt.subplot(1, 2, 1)
plt.hist(df['bytes_sent'], bins = 50,
         color = 'c', edgecolor = 'black')
plt.title('Исходное распределение')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.subplot(1, 2, 2)
plt.hist(df['bytes_robust'], bins = 50,
         color = 'c', edgecolor = 'black')
plt.title('Распредление после RobustScaler')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.tight_layout(pad = 1)
```



Результат масштабирования менее интерпретируем, чем Z-score.

Для StandardScaler Z-score имеет четкую вероятностную интерпретацию (для нормального распределения):

$$Z = 2 \Longrightarrow 97.7\text{-й процентиль}.$$

Для RobustScaler интерпретация менее интуитивна:

$$r = 2 \Longrightarrow \text{«на 2 } IQR \text{ выше медианы»}.$$

Что это означает в процентилях? Зависит от распределения.

- для нормального: $r = 2 \approx 97.7\text{-й процентиль}$;
- для экспоненциального: $r = 2 \approx 95\text{-й процентиль}$;
- для равномерного: $r = 2 \approx 100\text{-й процентиль}(\text{максимум})$.

Это приводит к тому, что аналитик SOC, видя $r = 2$, не может мгновенно понять, насколько это «редкое» событие. Ему нужно смотреть на распределение признака.

Поэтому можно использовать RobustScaler для алгоритмов, но для интерпретации возвращаться к исходным значениям или использовать процентили.

Так же как и StandardScaler, RobustScaler не работает для разреженных данных.

In [39]:
```
import numpy as np
from scipy.sparse import csr_matrix
# Создаем разреженную матрицу (имитация TF-IDF) - 70% нулей
data = np.array([[1, 0],
                 [0, 2],
                 [3, 0],
                 [0, 0],
                 [2, 0]])
sparse_data = csr_matrix(data)

print(f"Исходная разреженность: {(sparse_data == 0).mean():.1%}\n")

robust_scaler = RobustScaler()
dense_robust = robust_scaler.fit_transform(sparse_data.toarray())
print("После RobustScaler:")
```

```
print(np.round(dense_robust, 2))
print(f"Разреженность: {(dense_robust == 0).mean():.1%}\n")
```

```
Исходная разреженность: 60.0%

После RobustScaler:
[[ 0.   0. ]
 [-0.5  2. ]
 [ 1.   0. ]
 [-0.5  0. ]
 [ 0.5  0. ]]
Разреженность: 50.0%
```

MinMaxScaler

Этот метод масштабирования приводит данные к заданному диапазону (по умолчанию от 0 до 1). Для того, чтобы сделать масштабирование данных необходимо для каждого признака найти его минимум (min) и максимум (max), а затем применить следующее преобразование:

$$mm_i = \frac{x_i - x_i^{min}}{x_i^{max} - x_i^{min}}.$$

Результат преобразования:

- Все значения попадают в диапазон [0, 1].
- Минимальное значение становится 0.
- Максимальное значение становится 1.

При необходимости можно привести данные к произвольному диапазону $[a, b]$:

$$mm_i = a + \frac{(x_i - x_i^{min}) \cdot (b - a)}{x_i^{max} - x_i^{min}}.$$

На первый взгляд, это самый простой и логичный способ масштабирования. Но в кибербезопасности он таит в себе фатальную ловушку, которая может полностью "ослепить" вашу модель. Давайте разберем, почему это происходит и где этот метод всё-таки применим.

Преимущества MinMaxScaler.

1. Строгий диапазон [0, 1]. Это критически важно для алгоритмов, которые математически не могут работать с числами вне этого диапазона. Например, нейросети с функцией активации Sigmoid. Если подать на вход Sigmoid число 1000, она вернет 1.0, и градиент станет равен нулю (проблема насыщения). MinMax гарантирует, что входы будут в безопасной зоне.
2. Сохранение точных пропорций и формы распределения. В отличие от RobustScaler или StandardScaler, MinMax использует только крайние точки. Если

два значения отличались друг от друга в 2 раза до масштабирования, они будут отличаться в 2 раза и после (в абсолютных значениях разницы). Гистограмма признака просто сжимается по оси абсцис.

3. Интерпретируемость. Значение $mm = 0.8$ всегда означает: «это значение находится на уровне 80% от максимального зафиксированного в обучающей выборке». Это понятно для аналитиков.
4. Обработка константных признаков. Если признак не меняется (дисперсия равна 0), $x_{max} = x_{min}$. В `scikit-learn` `MinMaxScaler` элегантно обрабатывает этот edge-case: он просто присваивает таким признакам значение 0, избегая деления на ноль.

Недостатки MinMaxScaler.

Основным недостатком MinMaxScaler является чувствительность к выбросам.

Рассмотрим к каким признакам в нашем наборе данных можно применить MinMaxScaler, а к каким нельзя.

In [40]:
```
df.head(2)
```

Out[40]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523

Большинство признаков нашего набора имеют выбросы. Поэтому применение для их масштабирования MinMaxScaler приведет к эффекту «сжатия данных».

Покажем это на примере признака `bytes_sent`. Этот признак имеет выбросы (аномалии), которые соответствуют DDoS-атакам.

In [41]:
```
plt.figure(figsize = (10, 4))
plt.title('Распределение признака "bytes_sent"')
plt.hist(df['bytes_sent'], bins = 50,
         color = 'c', edgecolor = 'black')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений')
plt.grid()
```



In [42]:
```
df['bytes_sent'].describe(percentiles=[0.95])
```

Out[42]:

	bytes_sent
count	5.000000e+02
mean	4.981326e+05
std	8.594268e+06
min	1.113661e+00
50%	2.641097e+03
95%	2.886234e+05
max	1.915556e+08

dtype: float64

Большинство значений признака (475 наблюдений) лежит в диапазоне $[1.1, \sim 300\ 000]$.

DDoS-выбросы достигают ~200 000 000.

Применим для признака `bytes_sent` MinMaxScaler.

Для этого можно использовать класс `MinMaxScaler` модуля `preprocessing` библиотеки `sklearn`.

Параметр `feature_range` позволяет задавать требуемый диапазон изменения признака (по умолчанию `feature_range = (0, 1)`).

Посмотрим чему равно масштабированное значение для:

- медианного трафика (2 641 байт);
- крупной, но легитимной сессии, где `bytes_sent = 100 000` байт;
- очень крупная, но все еще нормальной сессия (95-й перцентиль, 288 623 байта).

In [43]:
```
from sklearn.preprocessing import MinMaxScaler
minmax = MinMaxScaler(feature_range = (0, 1))
# обучаем: считаем min и max
minmax_scaled = minmax.fit(df[['bytes_sent']])

x_median = np.array([[2641]])
x_large = np.array([[100000]])
x_95 = np.array([[288623]])

x_scaled_median = minmax_scaled.transform(x_median)
x_scaled_large = minmax_scaled.transform(x_large)
x_scaled_95 = minmax_scaled.transform(x_95)

print(f"Исходное: {x_median[0,0]} → Масштабированное: {x_scaled_median[0,0]:.4f}
print(f"Исходное: {x_large[0,0]} → Масштабированное: {x_scaled_large[0,0]:.4f}")
print(f"Исходное: {x_95[0,0]} → Масштабированное: {x_scaled_95[0,0]:.4f}")
```

```
Исходное: 2641 → Масштабированное: 0.0000
Исходное: 100000 → Масштабированное: 0.0005
Исходное: 288623 → Масштабированное: 0.0015
```

Что произошло?

Все 475 нормальных сессий (от 1 до 300 000 байт) будут сжаты в небольшой диапазон [0.0000, 0.0015].

Для алгоритма машинного обучения (особенно нейросети или KNN) значения 0.00008, 0.0025 и 0.0015 практически неотличимы от нуля. Модель потеряет способность различать абсолютно весь легитимный трафик, от маленького HTTP-запроса до скачивания файла на 180 КБ. Вся «норма» будет "сжата" в невидимую точку [0.0000, 0.0015], а 25 DDoS-выбросов будут иметь масштабированные значения около 1.0.

Исходя из описанных преимуществ и недостатков MinMaxScaler целесообразно применить к признакам `entropy` и `severity_score`.

Почему `entropy` — идеальный кандидат для MinMaxScaler?

Причина 1: Наличие жестких физических границ.

Энтропия Шеннона для байтового потока (payload) математически ограничена.

- Минимум = 0 (все байты одинаковы, например, сплошные нули).
- Максимум = 8 бит на байт (идеальная случайность, поскольку в байте 256 значений, а $log_2(256) = 8$).

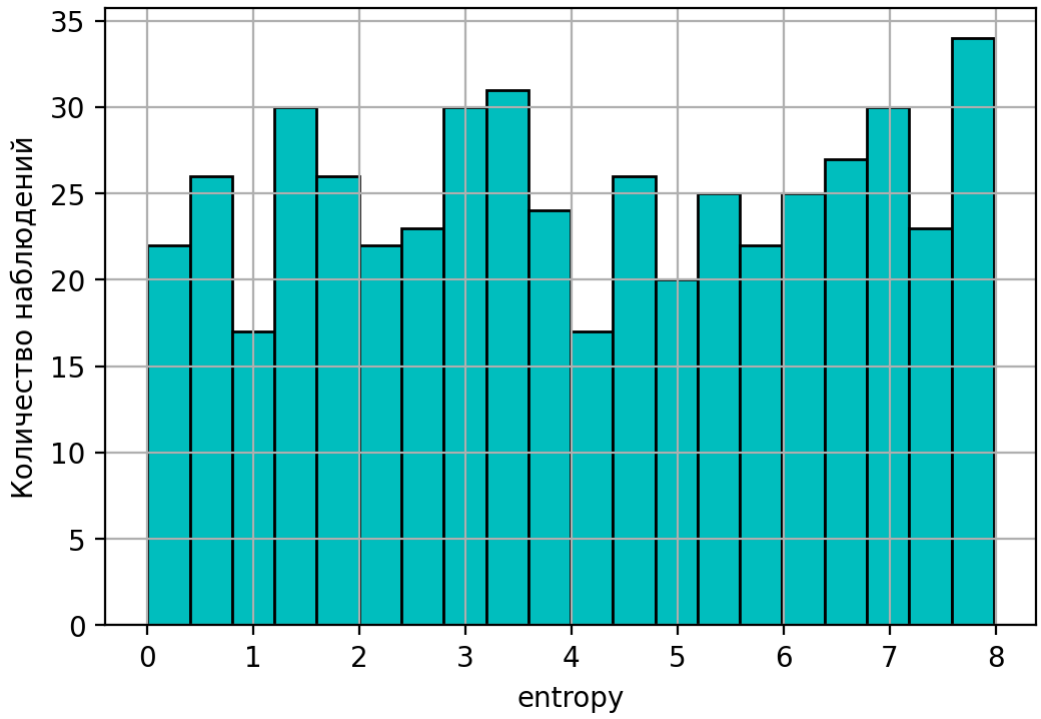
У этого признака есть естественный, физический диапазон [0, 8]. MinMaxScaler создан именно для таких случаев. Он элегантно и точно растянет этот диапазон до [0, 1].

Причина 2: Отсутствие экстремальных выбросов.

Признак имеет равномерное распределение. Здесь нет «тяжелых хвостов». Максимальное значение в выборке будет очень близко к теоретическому

максимуму (8). Следовательно, знаменатель в формуле $(x_{max} - x_{min})$ будет равен примерно 8. Никакого «сжатия» нормальных данных не произойдет.

In [44]:
```
plt.figure(figsize = (6, 4))
plt.hist(df['entropy'], bins = 20,
         color = 'c', edgecolor = 'black')
plt.xlabel('entropy')
plt.ylabel('Количество наблюдений')
plt.grid()
```



Посмотрим чему равны масштабированные значения для нескольких промежуточных значений признака.

In [46]:
```
from sklearn.preprocessing import MinMaxScaler
minmax = MinMaxScaler(feature_range = (0, 1))
minmax_scaled = minmax.fit(df[['entropy']]) # обучаем: считаем min и max

x_25 = df['entropy'].quantile(0.25)
x_50 = df['entropy'].median()
x_75 = df['entropy'].quantile(0.75)
x_95 = df['entropy'].quantile(0.95)

x_scaled_25 = minmax_scaled.transform(np.array(x_25).reshape(-1, 1))
x_scaled_50 = minmax_scaled.transform(np.array(x_50).reshape(-1, 1))
x_scaled_75 = minmax_scaled.transform(np.array(x_75).reshape(-1, 1))
x_scaled_95 = minmax_scaled.transform(np.array(x_95).reshape(-1, 1))

print(f"Исходное: {x_25} → Масштабированное: {x_scaled_25[0,0]:.14f}")
print(f"Исходное: {x_50} → Масштабированное: {x_scaled_50[0,0]:.4f}")
print(f"Исходное: {x_75} → Масштабированное: {x_scaled_75[0,0]:.4f}")
print(f"Исходное: {x_95} → Масштабированное: {x_scaled_95[0,0]:.4f}")
```

```
Исходное: 2.0183310997686403 → Масштабированное: 0.25248830996960
Исходное: 3.9740802141907956 → Масштабированное: 0.4977
Исходное: 6.219382539593137 → Масштабированное: 0.7792
Исходное: 7.705193558521901 → Масштабированное: 0.9655
```

Все значения признака равномерно легли в диапазон $[0, 1]$.

Ограничения MinMaxScaler.

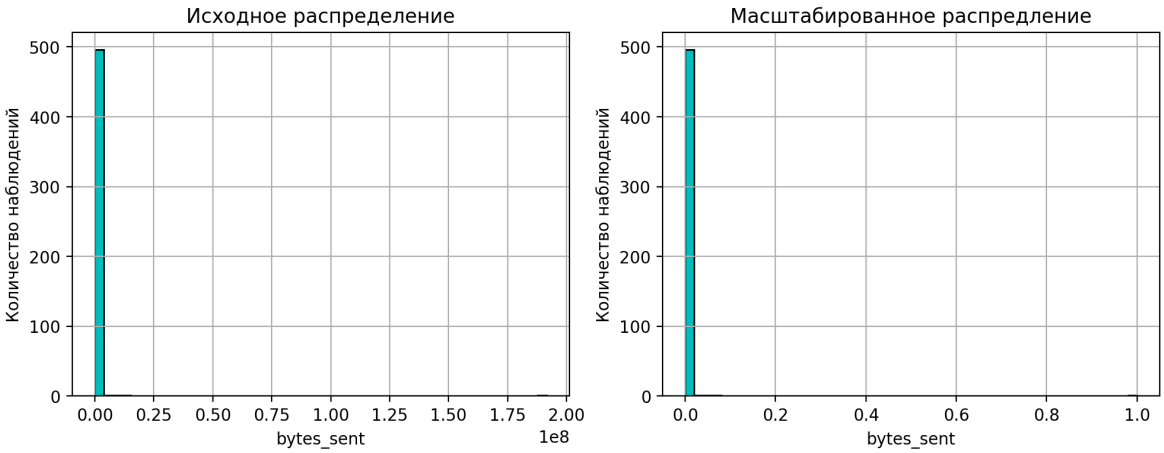
Не меняет форму распределения.

MinMaxScaler — это линейное преобразование. Оно только умножает все значения на константу и сдвигает их.

Если исходный признак `bytes_sent` имел логнормальное распределение с тяжелым хвостом, то после MinMax он останется логнормальным распределением с тяжелым хвостом, просто сжатым в диапазон $[0, 1]$. Алгоритмы, требующие нормальности, всё равно будут работать плохо.

Сравнение распределений признака до и после масштабирования.

In [47]:
```
plt.figure(figsize = (10, 4))
plt.subplot(1, 2, 1)
plt.hist(df['bytes_sent'], bins = 50,
         color = 'c', edgecolor = 'black')
plt.title('Исходное распределение')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.subplot(1, 2, 2)
plt.hist(minmax.fit_transform(df[['bytes_sent']]), bins = 50,
         color = 'c', edgecolor = 'black')
plt.title('Масштабированное распредление')
plt.xlabel('bytes_sent')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.tight_layout(pad = 1)
```



Уничтожает разреженность данных.

Как было показано выше, любая операция вычитания уничтожает разреженность. Если $x_{min} \neq 0$ (а в реальных данных это почти всегда так), то исходные нулевые значения превратятся в нули.

Следовательно, MinMaxScaler категорически нельзя применять к разреженным матрицам (TF-IDF, One-Hot Encoding).

Зависимость от границ обучающей выборки.

Поскольку масштабирование жестко привязано к x_{min} и x_{max}, что будет, если в тестовой выборке появится значение, большее, чем максимум в обучающей?

Например, $x_{max_{train}} = 1000$, а в тестовых данных есть значение $x_{test} = 1200$. Тогда для него масштабированное значение будет равно 1.1, что выходит за пределы диапазона $[0, 1]$. Для линейной регрессии, методов расстояний и нейросетей это может быть критично.

Примечание: в `scikit-learn` можно использовать параметр `clip=True` в `MinMaxScaler`, чтобы принудительно обрезать значения > 1 до 1, но это всё равно искажает информацию.

MaxAbsScaler

У всех рассмотренных методов - MinMax, Standard и Robust - есть одна общая черта, о которой мы говорили в контексте разреженных данных. Все они выполняют операцию ЦЕНТРИРОВАНИЯ (вычитают минимум, среднее или медиану).

А что, если вычитание — это именно то, что нам запрещено делать? Что, если в наших данных миллионы нулей и превращение их в ненулевые значения "убьет" нашу программу из-за нехватки памяти?

Для масштабирования разреженных данных применяется метод MaxAbsScaler.

Метод масштабирует данные, приводя их к диапазону $[-1, 1]$ на основе выражения вида:

$$ma_i = \frac{x_i}{max|x_i|}.$$

Результат преобразования:

Если все данные были положительными, диапазон будет [0, 1].

Если все данные были отрицательными, диапазон будет [−1, 0],

Если были и положительные, и отрицательные, диапазон будет [−1, 1].

Мы не сдвигаем данные. Мы не вычитаем из них никакие константы. Мы просто делим каждое значение на самое большое число в столбце. Это чистое масштабирование (изменение масштаба линейки), без центрирования.

Для демонстрации MaxAbsScaler рассмотрим признак `entropy`: его значения ограничены и в созданном датасете не имеют экстремальных выбросов.

In [ ]:
```
df.head(2)
```

Out[ ]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523

Посмотрим чему равны масштабированные значения для нескольких промежуточных значений признака.

In [ ]:
```
ma = MaxAbsScaler()
ma_scaled = ma.fit(df[['entropy']]) # обучаем: считаем abs(max)

x_25 = df['entropy'].quantile(0.25)
x_50 = df['entropy'].median()
x_75 = df['entropy'].quantile(0.75)
x_95 = df['entropy'].quantile(0.95)

x_scaled_25 = ma_scaled.transform(np.array(x_25).reshape(-1, 1))
x_scaled_50 = ma_scaled.transform(np.array(x_50).reshape(-1, 1))
x_scaled_75 = ma_scaled.transform(np.array(x_75).reshape(-1, 1))
x_scaled_95 = ma_scaled.transform(np.array(x_95).reshape(-1, 1))

print(f"Исходное: {x_25} → Масштабированное: {x_scaled_25[0,0]:.14f}")
print(f"Исходное: {x_50} → Масштабированное: {x_scaled_50[0,0]:.4f}")
print(f"Исходное: {x_75} → Масштабированное: {x_scaled_75[0,0]:.4f}")
print(f"Исходное: {x_95} → Масштабированное: {x_scaled_95[0,0]:.4f}")
```

```
Исходное: 2.0183310997686403 → Масштабированное: 0.25292117859245
Исходное: 3.9740802141907956 → Масштабированное: 0.4980
Исходное: 6.219382539593137 → Масштабированное: 0.7794
Исходное: 7.705193558521901 → Масштабированное: 0.9656
```

Главное преимущество MaxAbsScaler - сохранение разреженности данных (Sparsity). Ради него он и был создан.

В задачах обработки текстов (TF-IDF) или категориального кодирования (One-Hot Encoding) мы получаем разреженные матрицы (sparse matrices). Это матрицы, где 95–99% значений — это нули.

Хранить их в виде обычной таблицы (dense array) — это трата гигантского объема оперативной памяти. Поэтому используются специальные форматы (например, scipy. `sparse.csr_matrix`), которые хранят только ненулевые элементы и их индексы.

При использовании MinMaxScaler он пытается вычислить min и max по каждому признаку. Для этого он делает операции, которые в текущей реализации `sklearn` требуют плотного представления. В результате разреженная матрица, например, 100 000 документов × 200 000 токенов, превращается в плотную — это $2 \cdot 10^{10}$ чисел. Даже если каждое число занимает 4 байта, это уже ~80 ГБ памяти.

Для реализации метода MaxAbsScaler используется класс `MaxAbsScaler` модуля `preprocessing` библиотеки `sklearn` . Этот класс нативно поддерживает

`scipy.sparse` матрицы и обрабатывает их максимально эффективно.

In [48]:
```
from scipy.sparse import csr_matrix
from sklearn.preprocessing import MaxAbsScaler

# Разреженная матрица (имитация TF-IDF для логов)
# 80% нулей
sparse_data = csr_matrix([[0, 0, 5],
                          [0, 3, 0],
                          [0, 0, 0],
                          [2, 0, 0]])

print(f"Исходная разреженность: {(sparse_data == 0).mean():.1%}")

# MaxAbsScaler работает с sparse напрямую
maxabs = MaxAbsScaler()
sparse_scaled = maxabs.fit_transform(sparse_data)

print(f"Разреженность после MaxAbs: {(sparse_scaled == 0).mean():.1%} - все 0 со
```

```
Исходная разреженность: 75.0%
Разреженность после MaxAbs: 75.0% - все 0 сохранены.
```

Ограничения MaxAbsScaler.

Чувствительность к выбросам.

Поскольку мы делим на $max(|\ x\ |)$, один экстремальный выброс «растянет» знаменатель и сожмет все нормальные значения в ноль.

Пример:

Нормальный трафик: 1000 байт. DDoS-выброс: 10 000 000 байт.

$$ma_{norm} = \frac{1000}{10000000} = 0.0001.$$

Вся норма "схлопнулась" в невидимость.

Отсутствие центрирования.

MaxAbsScaler не делает среднее равным нулю.

Пример:

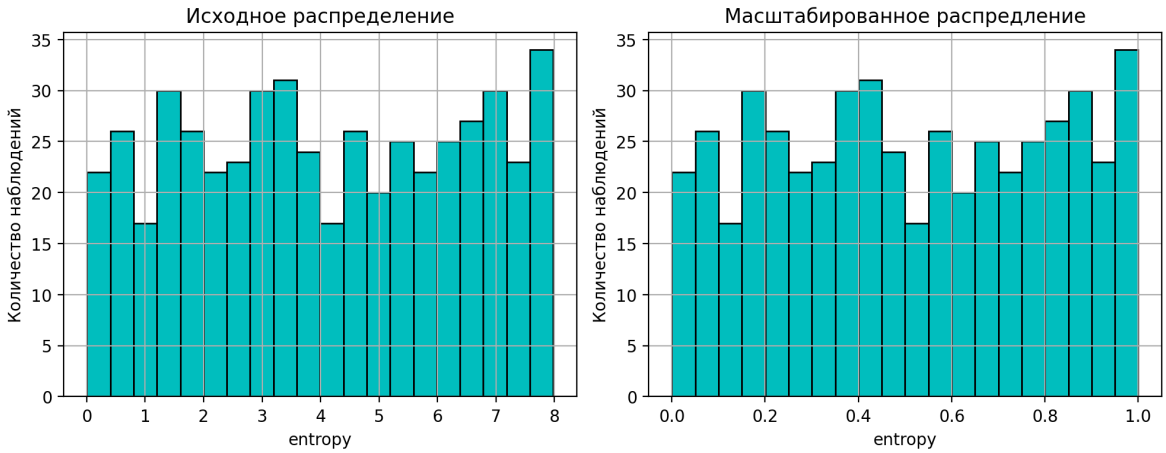
Предположим есть данные [1, 2, 3, 4, 5]. После деления получится [0.2, 0.4, 0.6, 0.8, 1.0]. Среднее этих значений = 0.6, а не 0.

Алгоритмы, которые математически ожидают центрированные данные (например, PCA или методы, использующие L2-регуляризацию без учета сдвига), будут работать некорректно или сходиться медленнее.

Не изменяет форму исходного распределения.

In [55]:
```
plt.figure(figsize = (10, 4))
plt.subplot(1, 2, 1)
```

```
plt.hist(df['entropy'], bins = 20,
         color = 'c', edgecolor = 'black')
plt.title('Исходное распределение')
plt.xlabel('entropy')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.subplot(1, 2, 2)
plt.hist(ma.fit_transform(df[['entropy']]), bins = 20,
         color = 'c', edgecolor = 'black')
plt.title('Масштабированное распредление')
plt.xlabel('entropy')
plt.ylabel('Количество наблюдений')
plt.grid()
plt.tight_layout(pad = 1)
```



L2-нормализация

Этот метод масштабирования основан на нормализации не столбцов, а строк. Каждая строка делится на свою собственную L2-норму:

$$L = \frac{x}{\|x\|_2} = \frac{x}{\sqrt{\sum_{i=1}^{n} x_i^2}}.$$

В результате каждый вектор признаков превращается в единичный вектор на поверхности гиперсферы. После нормализации все наблюдения находятся на одинаковом «расстоянии» от начала координат, и различия между ними определяются только направлением (углом), а не длиной.

Для нормализации наблюдений используется класс `Normalizer` модуля `preprocessing` библиотеки `sklearn`.

Рассмотрим пример L2-нормализации для нашего набора данных.

In [53]:
```
df.head(2)
```

Out[53]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523

In [56]:

```
from sklearn.preprocessing import Normalizer

# Исходные количественные признаки, описывающие сетевую сессию.
feature_columns = ["duration", "bytes_sent",
                   "n_packets", "response_time",
                   "entropy"]

# L2-нормализация выполняется отдельно для каждой строки:
# все значения строки делятся на её L2-норму.
normalizer_l2 = Normalizer(norm = "l2")
normalized_values = normalizer_l2.fit_transform(df[feature_columns])

# Передаём имена и индексы исходных данных.
df_l2 = pd.DataFrame(normalized_values,
                     columns = feature_columns,
                     index = df.index)

print("После L2-нормализации:")
display(df_l2.describe().round(4))

# Проверяем длину каждого полученного вектора (строки).
row_norms = np.linalg.norm(df_l2.to_numpy(), axis=1)
print(f"Нормы строк: min={row_norms.min():.4f}, "
      f"max={row_norms.max():.4f}")
```

```
После L2-нормализации:
```

	duration	bytes_sent	n_packets	response_time	entropy
count	500.0000	500.0000	500.0000	500.0000	500.0000
mean	0.0019	0.8582	0.0337	0.2763	0.0040
std	0.0038	0.2596	0.0668	0.3383	0.0065
min	0.0000	0.0040	0.0000	0.0000	0.0000
25%	0.0001	0.8723	0.0018	0.0147	0.0001
50%	0.0004	0.9950	0.0118	0.0954	0.0010
75%	0.0019	0.9999	0.0517	0.4754	0.0049
max	0.0401	1.0000	0.8840	0.9970	0.0431

```
Нормы строк: min=1.0000, max=1.0000
```

Достоинства:

- Идеален для задач, где важна косинусная близость (cosine similarity), а не евклидово расстояние. Например, сравнение профилей поведения пользователей (UEBA — User and Entity Behavior Analytics).

- Устраняет влияние «масштаба активности». Если один хост генерирует в 100 раз больше трафика, чем другой, просто потому что он сервер, L2-нормализация уберет эту разницу и позволит сравнивать паттерны поведения, а не объемы.

Недостатки:

- Теряет информацию об абсолютном масштабе. В кибербезопасности масштаб часто критичен. DDoS-атака — это именно огромный объем трафика. Если мы нормализуем строки, DDoS-сессия и обычная сессия могут выглядеть одинаково, потому что их «направления» (соотношение признаков) похожи, а разница в объеме исчезла.
- Не подходит для признаков с разной семантикой в одной строке (порт + байты + энтропия).

Когда использовать в кибербезопасности:

- Для UEBA (User and Entity Behavior Analytics), когда сравниваются векторы поведения пользователей.
- Для текстовых логов после TF-IDF векторизации (косинусная близость документов).
- Для эмбеддингов (например, векторных представлений IP-адресов или доменов).

Когда НЕ использовать:

Для классического обнаружения вторжений (IDS), где абсолютные значения признаков (`bytes_sent` , `n_packets`) несут критическую информацию об атаке.

2. Нелинейные преобразования признаков

Мы разобрали методы линейного масштабирования: MinMax, Standard, Robust, MaxAbs. И во всех этих методах есть одна общая черта: они не меняют форму распределения. Они только сжимают, растягивают или сдвигают данные, но «рисунок» на гистограмме остается тем же самым.

На практике в кибербезопасности большинство признаков имеют «неправильную» форму распределения:

- `bytes_sent` — логнормальное распределение с тяжелым правым хвостом;
- `duration` — экспоненциальное распределение;
- `n_packets` — распределение Пуассона.

При этом «неправильная» форма — не просто эстетическая проблема. Она снижает качество алгоритмов машинного обучения.

Цели нелинейных преобразований.

1. Требования алгоритмов к нормальности.

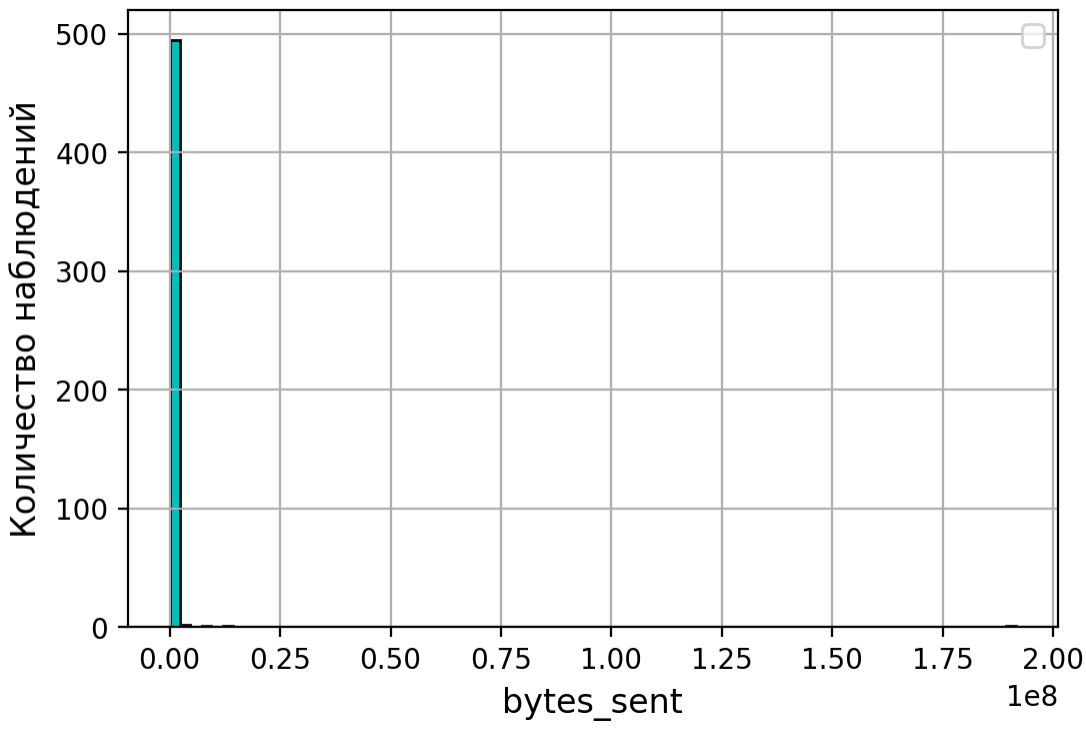
Многие классические алгоритмы машинного обучения математически предполагают, что входные данные имеют нормальное (или хотя бы симметричное) распределение.

- Линейная и логистическая регрессии.
- LDA (Linear Discriminant Analysis). Метод классификации, который строит разделяющие гиперплоскости, предполагая нормальность внутри каждого класса.
- Gaussian Naive Bayes. Название говорит само за себя — он буквально предполагает гауссово распределение признаков.
- PCA (Principal Component Analysis). Метод снижения размерности, который максимизирует дисперсию. На скошенных данных PCA будет строить компоненты, искаженные асимметрией.

2. Проблема «тяжелых хвостов».

Рассмотрим распределение признака `bytes_sent`.

In [57]:
```
plt.figure(figsize = (6, 4))
plt.hist(df['bytes_sent'],
         color = 'c',
         bins = 80,
         edgecolor = 'black',
         rwidth = 0.9)
plt.xlabel('bytes_sent', fontsize = 12)
plt.ylabel('Количество наблюдений', fontsize = 12)
plt.legend()
plt.grid()
```



Это пример распределения с "тяжелым хвостом".

Почему это проблема?

Большинство алгоритмов машинного обучения (линейные модели, KNN, SVM, нейросети) работают с абсолютными различиями между значениями.

Абсолютная разница между сессией на 1000 байт и сессией на 2000 байт составляет 1000 байт. Относительная разница равна 2 (рост трафика на 100%). Для аналитика это может быть важным сигналом (например, начало эксфильтрации).

Абсолютная разница между сессией на 10 000 000 байт и сессией на 10 001 000 байт так же равна 1000 байт. Но относительная разница равна 1.001 (рост трафика на 0.01%). Абсолютно несущественно.

Для алгоритма, работающего с абсолютными значениями, оба сценария выглядят абсолютно одинаково. Разница везде 1000 байт. Модель не может отличить:

- Важный сигнал (удвоение трафика — возможно, эксфильтрация)
- От неважного шума (микроскопическое колебание в большом потоке).

Это так называемая «слепота к малым различиям»: алгоритм видит только абсолютные числа и не понимает, что относительное изменение в 2 раза на малых значениях — это гораздо более важный сигнал, чем такое же абсолютное изменение на больших значениях.

3. Улучшение сходимости градиентных методов.

Нейросети и градиентный бустинг работают намного быстрее и стабильнее, когда распределение признаков симметрично. Скошенные данные создают «овраги» в функции потерь, и градиентному спуску приходится делать множество лишних итераций, чтобы найти минимум.

Существует достаточное большое количество методов нелинейных преобразований, основными из которых являются:

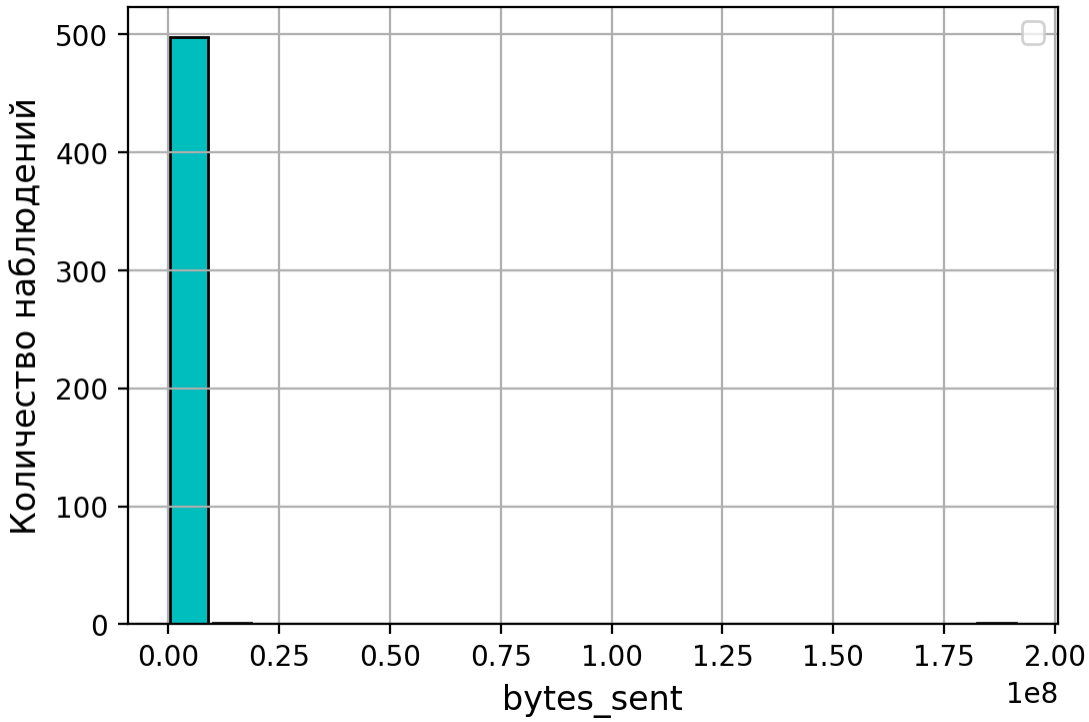
- логарифм;
- квадратный корень;
- Box-Cox;
- Yeo-Johnson;
- квантильное;
- биннинг (дискретизация).

Логарифмическое преобразование

В кибербезопасности большинство реальных признаков имеют рапспределения отличные от нормального, как правило, с "тяжелыми хвостами", скошенными вправо. Например, признак `bytes_sent`.

In [58]:
```
plt.figure(figsize = (6, 4))
plt.hist(df['bytes_sent'],
```

```
        color = 'c',
        bins = 20,
        edgecolor = 'black',
        rwidth = 0.9)
plt.xlabel('bytes_sent', fontsize = 12)
plt.ylabel('Количество наблюдений', fontsize = 12)
plt.legend()
plt.grid()
```



Это так называемое **логнормальное распределение**.

Для преобразования таких признаков используется логарифмическая функция:

$$X^{'} = log_a X,$$

a — основание логарифма ($a > 0$, $a \neq 1$).

Выбор основания логарифма осуществляется экспериментальным путем.

Как логарифм позволяет устранить проблему «тяжелых хвостов»?

Ключевое свойство логарифма:

Если в исходном пространстве признак изменился в k раз (мультипликативное изменение): $x_2 = k \cdot x_1$, то в логарифмическом пространстве разница между ними будет константой, не зависящей от абсолютных значений:

$$log(x_2) - log(x_1) = log(k \cdot x_1) - log(x_1) = log(k) + log(x_1) - log(x_1) = log(k).$$

Это означает, что мультипликативные изменения в исходных данных становятся аддитивными после логарифмирования.

Пример.

Сценарий А (подозрительная активность):

Если `bytes_sent` вырос с 1000 до 2000 (в 2 раза), то:

$$log_{10}(1000) = 3,\ log_{10}(2000) = 3.301.$$

Разница в лог-пространстве: $3.301 - 3 = 0.301$.

Сценарий Б (шумная активность):

Если `bytes_sent` вырос с 10 000 000 до 100 001 000 (в 1.0001 раз):

$$log_{10}(10000000) = 7,\ log_{10}(100010000) = 8.$$

Разница в лог-пространстве: $7.00000004 - 7 = 0.00000004$.

Теперь алгоритм видит:

- Сценарий А: разница 0.301 (существенная)
- Сценарий Б: разница 0.00000004 (почти нулевая, шум)

Отношение между этими различиями: 0.301/0.00000004 ≈ 7 500 000.

То есть в лог-пространстве алгоритм в 7.5 миллионов раз сильнее реагирует на удвоение трафика на малых значениях, чем на микроскопическое колебание на больших. Именно так и должен думать аналитик кибербезопасности.

Прежде чем рассмотреть технологию логарифмического преобразования, остановимся на проблеме нуля и ее решении.

У логарифма есть фатальный недостаток: $log(0)$ не определен.

В кибербезопасности это критическая проблема. Что делать, если у нас есть сессия с `bytes_sent = 0` (например, TCP-рукопожатие, которое не завершилось передачей данных)?

Решение: использовать преобразование $x' = log(1 + x)$.

Это преобразование:

- Для $x = 0$ дает $log(1) = 0$. Ноль остается нулем.
- Для больших x ведет себя почти как обычный логарифм: $log(1 + x) \approx log(x)$.

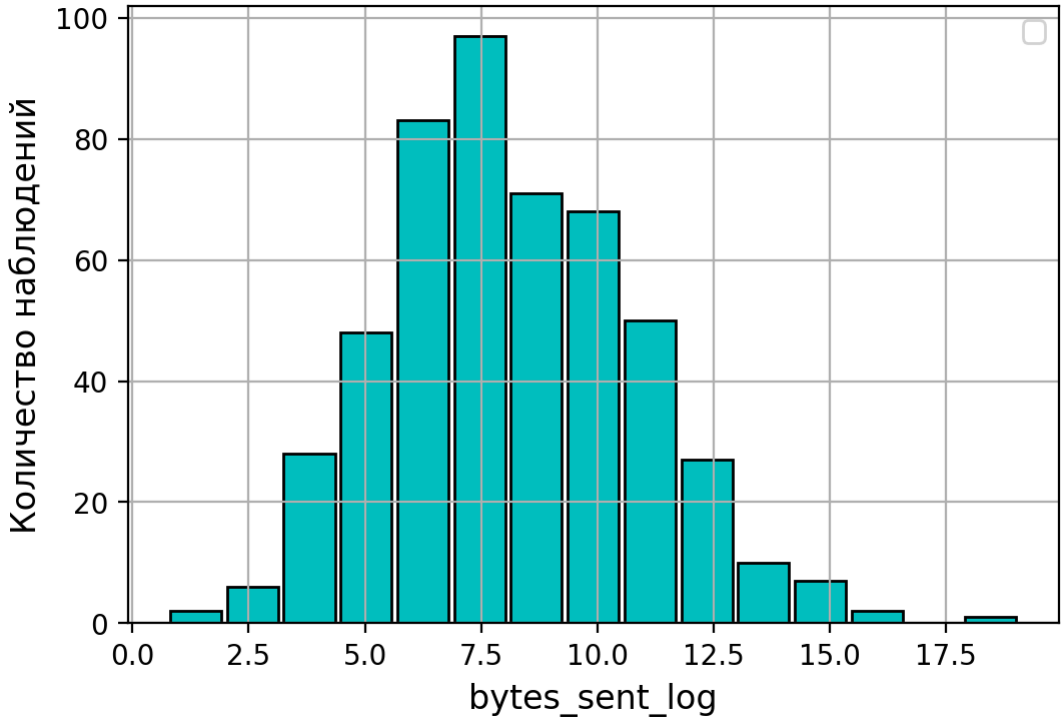
В `sklearn` и `numpy` есть специальная функция `log1p(x)`, которая вычисляет $log(1 + x)$ с повышенной точностью для малых x.

Рассмотрим пример применения логарифмического преобразования для признака `bytes_sent`.

In [59]:
```
plt.figure(figsize = (6, 4))

# Применяем логарифмическое преобразование
bytes_sent_log = np.log1p(df['bytes_sent'])
```

```
plt.hist(bytes_sent_log,
         color = 'c',
         bins = 15,
         edgecolor = 'black',
         rwidth = 0.9)
plt.xlabel('bytes_sent_log', fontsize = 12)
plt.ylabel('Количество наблюдений', fontsize = 12)
plt.legend()
plt.grid()
```



Оценим асимметрию (skewness) до и после преобразования.

In [60]:
```
from scipy import stats
print("=== Асимметрия (skewness) до и после логарифмирования ===")
print()
skew_before = stats.skew(df['bytes_sent'])
skew_after = stats.skew(bytes_sent_log)
print(f"'bytes_sent' | До: {skew_before:7.2f} | После: {skew_after:7.2f} | Умень
```

```
=== Асимметрия (skewness) до и после логарифмирования ===

'bytes_sent' | До:   22.05 | После:    0.34 | Уменьшение в 64.5x
```

Влияние на выбросы.

Посмотрим на примере признака `bytes_sent`, как логарифмическое преобразование влияет на выбросы.

In [61]:
```
print("=== Влияние логарифма на выбросы (bytes_sent) ===")
print()
print(f"Медиана (исходные): {df['bytes_sent'].median():.0f}")
print(f"Максимум (исходные): {df['bytes_sent'].max():.0f}")
print(f"Отношение max/median: {df['bytes_sent'].max() / df['bytes_sent'].median(

print(f"\nМедиана (log1p): {bytes_sent_log.median():.2f}")
```

```
print(f"Максимум (log1p): {bytes_sent_log.max():.2f}")
print(f"Отношение max/median: {bytes_sent_log.max() / bytes_sent_log.median():.2
```

```
=== Влияние логарифма на выбросы (bytes_sent) ===

Медиана (исходные): 2641
Максимум (исходные): 191555624
Отношение max/median: 72529x

Медиана (log1p): 7.88
Максимум (log1p): 19.07
Отношение max/median: 2.42x
```

В исходных данных максимум был в 72 529 раз больше медианы. Это экстремальный выброс.

После логарифмирования максимум стал всего в 2.42 раза больше медианы. Выброс всё еще виден, но он перестал быть экстремальным значением, которое доминирует над всеми остальными данными.

Это критически важно для алгоритмов, основанных на расстояниях (KNN, K-Means). После логарифмирования расстояния между точками становятся осмысленными, потому что выбросы больше не «растягивают» пространство признаков.

Комбинация: log1p + масштабирование.

Логарифмическое преобразование не заменяет масштабирование, а дополняет его.

Типичный пайплайн для признаков с тяжелыми хвостами:

- Логарифмирование (уменьшаем асимметрию, «выпрямляем» распределение).
- Масштабирование (приводим к сопоставимому диапазону).

In [62]:
```
# Пайплайн: log1p -> RobustScaler
bytes_sent_log = np.log1p(df[['bytes_sent']])

scaler = RobustScaler()
bytes_sent_final = scaler.fit_transform(bytes_sent_log)

print("=== Финальная статистика (log1p + RobustScaler) ===")
print(f"Медиана: {np.median(bytes_sent_final):.2f}")
print(f"Среднее: {np.mean(bytes_sent_final):.2f}")
print(f"Асимметрия: {stats.skew(bytes_sent_final)[0]:.2f}")
```

```
=== Финальная статистика (log1p + RobustScaler) ===
Медиана: 0.00
Среднее: 0.08
Асимметрия: 0.34
```

Получаем практически идеальный результат:

- Медиана = 0 (благодаря RobustScaler).
- Асимметрия = 0.45 (почти симметричное распределение, благодаря log1p).

Данные готовы для подачи в любой алгоритм.

Ограничения логарифмического преобразования.

1. Только для положительных значений.
2. Не всегда делает распределение нормальным. Логарифм уменьшает асимметрию, но не гарантирует нормальность.
3. Теряется интерпретируемость в исходных единицах. Например, после логарифмирования значение `bytes_sent_log = 10` не означает «10 байт». Это означает $e^{10} - 1 \approx 22025$ байт. Аналитикам нужно объяснять, что модель работает в «лог-пространстве».

Логарифмическое преобразование для лениаризации зависимостей.

До сих пор мы говорили о логарифме как о средстве «выпрямления» гистограммы одного признака. Но у логарифма есть ещё одна суперспособность, не менее важная для кибербезопасности: он умеет превращать нелинейные зависимости между признаками и целевой переменной в линейные.

Почему это критически важно? Потому что линейные модели (линейная регрессия, логистическая регрессия, Ridge, Lasso) — это не просто «простые» алгоритмы. Это интерпретируемые алгоритмы. В кибербезопасности интерпретируемость — это не роскошь, а требование. Когда модель помечает сессию как атаку, аналитик SOC должен понимать почему. Линейная модель даёт ответ: «потому что `bytes_sent` увеличился на X, и это дало вклад Y в оценку критичности». Нейросеть такого ответа не даст.

Но линейная модель работает только тогда, когда зависимость между признаком и целевой переменной действительно линейна. А на практике эти зависимости почти всегда нелинейные.

Логарифмирование одной или обеих переменных позволяет линеаризовать некоторые распространённые зависимости. Выбор преобразования зависит от предполагаемой формы связи; универсального преобразования для всех нелинейных зависимостей нет.

Исходная зависимость	Что преобразовать	Полученная линейная зависимость
Экспоненциальная: $y = Ae^{bx}$	y	$\ln y = \ln A + bx$
Степенная: $y = Ax^b$	и x, и y	$\ln y = \ln A + b \ln x$
Логарифмическая: $y = a + b \ln x$	x	Если $u = \ln x$, то $y = a + bu$

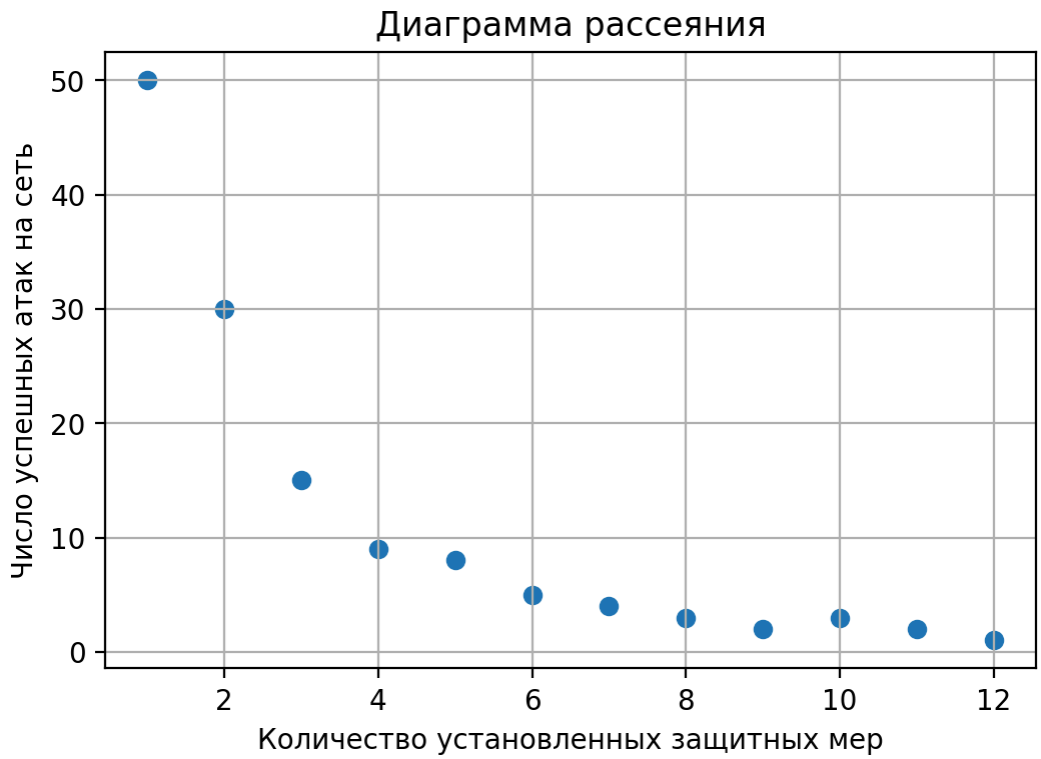
Рассмотрим пример линеаризации.

Предположим, задача заключается в прогнозировании числа успешных атак на сеть в зависимости от количества установленных защитных мер (например, количество активированных правил фаервола, число используемых антивирусных программ и т. д.).

Эмпирические данные могут показать, что зависимость между числом защитных мер (x) и количеством атак (y) имеет нелинейный характер: при малых значениях x снижение числа атак происходит быстро, но по мере увеличения - эффект насыщения приводит к замедлению снижения атак.

In [63]:
```
x = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
y = np.array([50, 30, 15, 9, 8, 5, 4, 3, 2, 3, 2, 1])

plt.figure(figsize = (6, 4))
plt.scatter(x, y)
plt.title('Диаграмма рассеяния')
plt.xlabel('Количество установленных защитных мер')
plt.ylabel('Число успешных атак на сеть')
plt.grid()
```



Зависимость между признаками носит нелинейный характер и может быть описана степенной функцией:

$$y = b_0 \cdot x^{b1}.$$

Логарифмирование исходных данных позволяет преобразовать нелинейную зависимость к линейному виду.

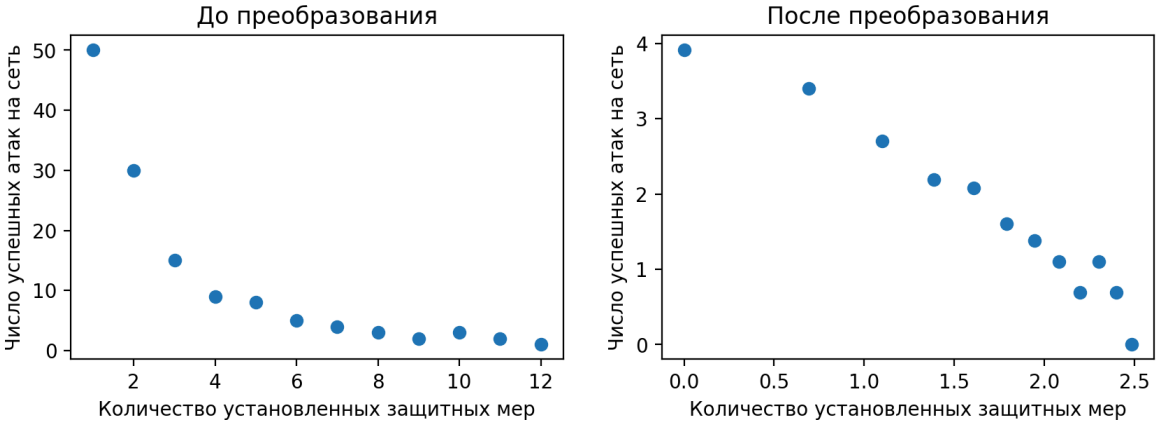
Для нашего примера необходимо прологарифмировать по основанию e обе части уравнения:

$$ln(y) = ln(b_0 \cdot x^{b1}) = ln(b_0) + ln(x^{b1}) = ln(b_0) + b_1 \cdot ln(x).$$

Полученное уравнение уже является линейным.

Визуализируем зависимости до и после логарифмического преобразования.

In [64]:
```
fig, axs = plt.subplots(nrows = 1, ncols = 2, figsize = (10,3))
axs[0].scatter(x, y)
axs[0].set_title('До преобразования')
axs[0].set_xlabel('Количество установленных защитных мер')
axs[0].set_ylabel('Число успешных атак на сеть')
plt.grid()
axs[1].scatter(np.log(x), np.log(y))
axs[1].set_title('После преобразования')
axs[1].set_xlabel('Количество установленных защитных мер')
axs[1].set_ylabel('Число успешных атак на сеть')
plt.grid();
```



Оценим степень зависимости до и после логарифмического преобразования.

In [65]:
```
print(f'Коэффициент корреляции до преобразования: {np.corrcoef(x, y)[0,1]}')
print(f'Коэффициент корреляции после преобразования: {np.corrcoef(np.log(x), np.
```

```
Коэффициент корреляции до преобразования: -0.7926432262958892
Коэффициент корреляции после преобразования: -0.9776126718004096
```

Для использования преобразованных моделей необходимо иметь возможность получать исходные значения параметров, т. е. применяемые нелинейные преобразования должны быть обратимыми.

Предположим, что после обучения получены параметры модели:

$$ln(b_0) = 2.3,\ b_1 = 0.8.$$

Чтобы получать прогнозные значения целевого признака y необходимо восстановить значение коэффициента b_0:

$$b_0 = e^{2.3} \approx 9.97.$$

Тогда исходная модель будет иметь вид:

$$y = 9.97 \cdot x^{0.8}.$$

Рассмотрим пример применения логарифмирования в задаче регрессии на нашем наборе данных.

In [66]:
```
df.head(2)
```

Out[66]:

	dst_port	duration	bytes_sent	n_packets	response_time	entropy	severity_score
0	3389	1.442375	547.412464	29	435.242056	0.165979	24.244770
1	80	1.411094	2017.685808	20	262.725010	5.909041	37.698523

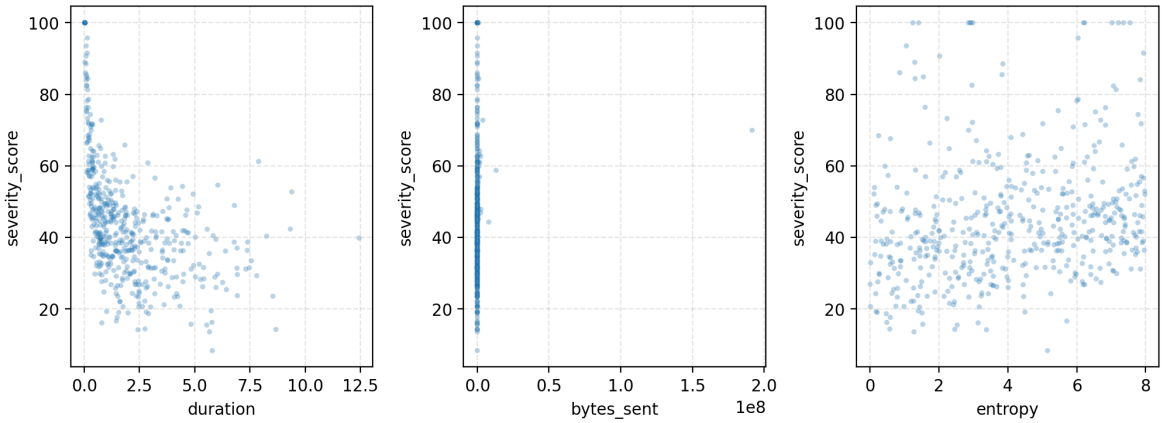
Предположим решается задача прогнозирования целевого признака `severity_score` (критичность инцидента) по признакам `duration`, `bytes_sent` и `entropy`.

Перед построением модели нужно провести визуальный анализ зависимости целевой переменной от каждого признака. С этой целью строятся диаграммы рассеяния.

In [67]:
```
pairs = [('duration', 'severity_score'),
         ('bytes_sent', 'severity_score'),
         ('entropy', 'severity_score')]

fig = plt.figure(figsize = (12, 4))
for i, (x_col, y_col) in enumerate(pairs):
    ax = plt.subplot(1, 3, i + 1)
    ax.scatter(df[x_col], df[y_col], alpha = 0.3, s = 10, edgecolors='none')
    ax.set_xlabel(x_col)
    ax.set_ylabel(y_col)
    ax.grid(True, linestyle='--', alpha=0.3)

plt.subplots_adjust(hspace = 0.4, wspace = 0.3);
```



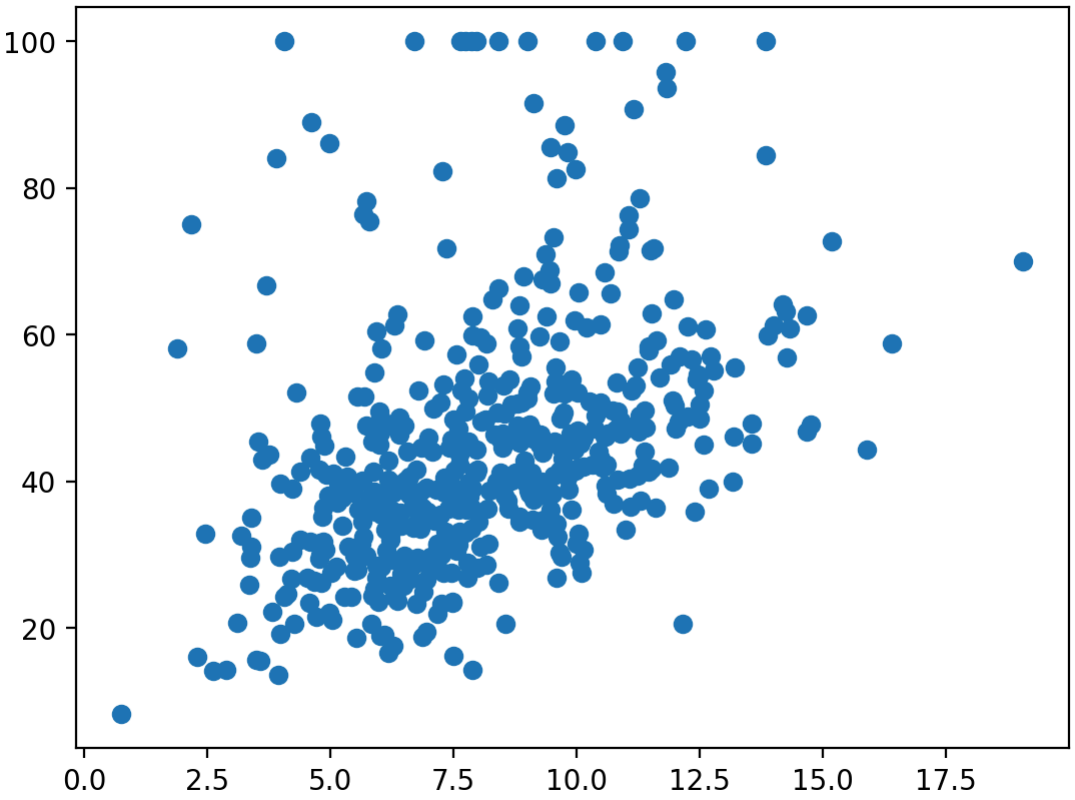
Из графиков видно:

- между признаками `duration` и `severity_score` есть нелинейная зависимость, которую можно описать обратной функцией $y \sim \frac{1}{x}$. В этом случае логарифмирование не нужно;
- между признаками `bytes_sent` и `severity_score` есть логарифмическая зависимость. Каждый дополнительный мегабайт трафика увеличивает критичность, но всё меньше и меньше. Первые 1000 байт дают огромный скачок, а разница между 10 ГБ и 11 ГБ — почти незаметна. Зависимость можно описать уравнением вида $y \sim \ln(x)$;

- между признаками `entropy` и `severity_score` есть слабая линейная зависимость, которую можно описать обратной функцией $y \sim x$. В этом случае логарифмирование не нужно.

Визуализируем зависимость между `severity_score` логарифмом `bytes_sent`.

In [68]:
```
plt.scatter(np.log1p(df['bytes_sent']), df['severity_score']);
```



Видно, что логарифмирование позволило получить линейную зависимость между признаками.

Покажем, что произойдет, если мы попытаемся предсказать `severity_score` с помощью линейной регрессии без логарифмирования `bytes_sent`:

$$\text{severity_score} = b_0 + b_1 \cdot \text{duration} + b_2 \cdot \text{bytes_sent} + b_3 \cdot \text{entropy}.$$

и с логарифмированием:

$$\text{severity_score} = b_0 + b_1 \cdot \text{duration} + b_2 \cdot \ln(\text{bytes_sent}) + b_3 \cdot \text{entropy}.$$

In [69]:
```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df1 = df.copy()

# Модель без преобразований
model_raw = LinearRegression()
model_raw.fit(df1[['bytes_sent', 'duration', 'entropy']], df1['severity_score'])
pred_raw = model_raw.predict(df1[['bytes_sent', 'duration', 'entropy']])
r2_raw = r2_score(df1['severity_score'], pred_raw)

# Модель с логарифмированием bytes_sent
```

```
df1['bytes_sent_log'] = np.log1p(df1['bytes_sent'])
X_log = df1[['bytes_sent_log', 'duration', 'entropy']]
model_log = LinearRegression()
model_log.fit(X_log, df1['severity_score'])
pred_log = model_log.predict(X_log)
r2_log = r2_score(df1['severity_score'], pred_log)

print("=== Сравнение линейных моделей ===")
print()
print(f"Без log(bytes_sent): R² = {r2_raw:.4f}")
print(f"С log(bytes_sent): R² = {r2_log:.4f}")
```

```
=== Сравнение линейных моделей ===

Без log(bytes_sent): R² = 0.2761
С log(bytes_sent): R² = 0.4374
```

Если учесть в модели гиперболическую зависимость признаков `duration` и `severity_score`, то качество модели значительно увеличивается.

In [70]:
```
# Модель без преобразований
model_raw = LinearRegression()
model_raw.fit(df1[['bytes_sent', 'duration', 'entropy']], df1['severity_score'])
pred_raw = model_raw.predict(df1[['bytes_sent', 'duration', 'entropy']])
r2_raw = r2_score(df1['severity_score'], pred_raw)

# Модель с логарифмированием bytes_sent и гиперболической зависимостью duration
df1['bytes_sent_log'] = np.log1p(df1['bytes_sent'])
df1['duration_reverse'] = 1.0 / (df1['duration'] + 0.1)
X_log_reverse = df1[['bytes_sent_log', 'duration_reverse', 'entropy']]
model_log_reverse = LinearRegression()
model_log_reverse.fit(X_log_reverse, df1['severity_score'])
pred_log_reverse = model_log_reverse.predict(X_log_reverse)
r2_log_reverse = r2_score(df1['severity_score'], pred_log_reverse)

print("=== Сравнение линейных моделей ===")
print()
print(f"Без log(bytes_sent): R² = {r2_raw:.4f}")
print(f"С log(bytes_sent): R² = {r2_log:.4f}")
print(f"С log(bytes_sent) и reverse(duration): R² = {r2_log_reverse:.4f}")
```

```
=== Сравнение линейных моделей ===

Без log(bytes_sent): R² = 0.2761
С log(bytes_sent): R² = 0.4374
С log(bytes_sent) и reverse(duration): R² = 0.8945
```

Логарифмическое преобразование полезно в следующих случаях:

1. Данные имеют скошенное вправо распределение (положительная асимметрия) - (положительный перекос) отличается тем, что хвост распределения тянется вправо, а пик смещается влевоправостороннее скошение (положительное перекосы). Логарифм помогает сделать распределение более близким к нормальному.
2. Нужно снизить влияние выбросов. Большие значения «сжимаются», что уменьшает их доминирование в модели.

3. Необходимо линеаризовать нелинейные зависимости между признаками. Например, моделирование ситуации, когда зависимость целевого признака от предикторов замедляется с ростом значения признака (эффект от увеличения инвестиций в кибербезопасность может снижаться по мере роста их объёма.

Преобразование Бокса-Кокса (Box-Cox)

Это преобразование является обобщением логарифмического преобразования.

Логарифмическое преобразование не подходит для некоторых видов исходных распределений (например, экспоненциальное). Это обусловлено тем, что скорость возрастания функции логрифма, равная $1/x$ не позволяет получить симметричные распределения.

Исходя из этого желательно иметь возможность изменения скорости возрастания функции λ. Тогда, меняя значение λ, можно подобарать оптимальное преобразование для любого исходного распределения.

Выражение для преобразования Box-Cox имеет следующий вид:

$$x^* = \begin{cases} \frac{x^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0 \\ log(x), & \text{if } \lambda = 0 \end{cases}$$

Павраметр λ подбирается методом максимального правдободобия.

Критерий оптимальности: алгоритм ищет λ, при котором:

- асимметрия (skewness) минимальна (близка к 0);
- эксцесс (kurtosis) близок к 0 (как у нормального распределения);
- тест Шапиро-Уилка даёт максимальный p-value.

Параметр λ определяет степень преобразования.

Значение λ	**Преобразование**	**Эффект**
$\lambda = 1$	$x' = x - 1$	Нет преобразования (линейный сдвиг)
$\lambda = 0.5$	$x' = 2(x - 1)$	Квадратный корень (умеренное «выпрямление»)
$\lambda = 0$	$x' = \ln(x)$	Логарифм (сильное «выпрямление»)
$\lambda = -0.5$	$x' = -2\left(\frac{1}{x} - 1\right)$	Обратный квадратный корень

Значение λ	Преобразование	Эффект
$\lambda = -1$	$x' = -\left(\frac{1}{x} - 1\right)$	Обратное значение (инверсия)

Преобразование Бокса-Кокса можно получить через класс `PowerTransformer` модуля `preprocessing` библиотеки `sklearn`.

По умолчанию данные нормализуются с нулевым средним и единичной дисперсией - параметр `standardize=True`.

Рассмотрим пример использования преобразования Бокса-Кокса на наших признаках и сравним его с обычным логарифмом.

In [72]:
```
from scipy.stats import shapiro
from sklearn.preprocessing import PowerTransformer

box_cox = PowerTransformer(method = 'box-cox')

# обучение модели (поиск оптимального параметра лямбда)
df1['bytes_sent_boxcox'] = box_cox.fit_transform(df[['bytes_sent']])
lambda_bytes = box_cox.lambdas_
df1['duration_boxcox'] = box_cox.fit_transform(df[['duration']])
lambda_duration = box_cox.lambdas_
df1['n_packets_boxcox'] = box_cox.fit_transform(df[['n_packets']])
lambda_packets = box_cox.lambdas_

print("=== Оптимальные значения λ ===")
print(f"bytes_sent: λ = {lambda_bytes[0]:.4f}")
print(f"duration: λ = {lambda_duration[0]:.4f}")
print(f"n_packets: λ = {lambda_packets[0]:.4f}")

# === СРАВНЕНИЕ С ЛОГАРИФМОМ ===
df1['bytes_sent_log'] = np.log1p(df1['bytes_sent'])
df1['duration_log'] = np.log1p(df1['duration'])
df1['n_packets_log'] = np.log1p(df1['n_packets'])

# === ОЦЕНКА НОРМАЛЬНОСТИ ===
from scipy.stats import skew, kurtosis

print("\n=== Асимметрия (skewness) ===")
print(f"{'Признак':15} | {'Исходные':>10} | {'log1p':>10} | {'Box-Cox':>10}")
print("-" * 55)
for col in ['bytes_sent', 'duration', 'n_packets']:
    skew_orig = skew(df1[col])
    skew_log = skew(df1[f'{col}_log'])
    skew_bc = skew(df1[f'{col}_boxcox'])
    print(f"{col:15} | {skew_orig:10.2f} | {skew_log:10.2f} | {skew_bc:10.2f}")

print("\n=== Эксцесс (kurtosis) ===")
print(f"{'Признак':15} | {'Исходные':>10} | {'log1p':>10} | {'Box-Cox':>10}")
print("-" * 55)
for col in ['bytes_sent', 'duration', 'n_packets']:
    kurt_orig = kurtosis(df1[col])
    kurt_log = kurtosis(df1[f'{col}_log'])
```

```
    kurt_bc = kurtosis(df1[f'{col}_boxcox'])
    print(f"{col:15} | {kurt_orig:10.2f} | {kurt_log:10.2f} | {kurt_bc:10.2f}")
```

```
=== Оптимальные значения λ ===
bytes_sent: λ = -0.0387
duration: λ = 0.2598
n_packets: λ = -1.4678

=== Асимметрия (skewness) ===
Признак         |   Исходные |      log1p |    Box-Cox
-----------------------------------------------------------
bytes_sent      |      22.05 |       0.34 |       0.00
duration        |       1.65 |       0.43 |      -0.04
n_packets       |       4.90 |       4.35 |       0.16

=== Эксцесс (kurtosis) ===
Признак         |   Исходные |      log1p |    Box-Cox
-----------------------------------------------------------
bytes_sent      |     487.45 |       0.20 |      -0.01
duration        |       3.09 |      -0.61 |      -0.37
n_packets       |      22.64 |      18.12 |       2.22
```

Оптимальные λ:

- `bytes_sent` : $\lambda = -0.0387$ — почти логарифм ($\lambda = 0$);
- `duration` : $\lambda = 0.2598$ — нечто между квадратным корнем ($\lambda = 0.5$) и логарифмом;
- `n_packets` : $\lambda = -1.4678$ — отрицательное значение, которое означает, что для распределения нужна инверсия.

Асимметрия:

- Box-Cox уменьшает асимметрию значительно лучше, чем логарифм: для `bytes_sent` и `duration` - идеальная симметрия.

Эксцесс:

- нормальное распределение имеет эксцесс ≈ 0. Box-Cox приближает эксцесс к 0 гораздо лучше, чем логарифм.

Сравним распределения признаков до и после преобразования.

In [74]:
```
# Список колонок для гистограмм
cols = ['bytes_sent', 'duration', 'n_packets',
        'bytes_sent_log', 'duration_log', 'n_packets_log',
        'bytes_sent_boxcox', 'duration_boxcox', 'n_packets_boxcox']

fig, axes = plt.subplots(3, 3, figsize = (14, 8))
axes = axes.ravel()  # превращаем двумерный массив осей в одномерный для удобной

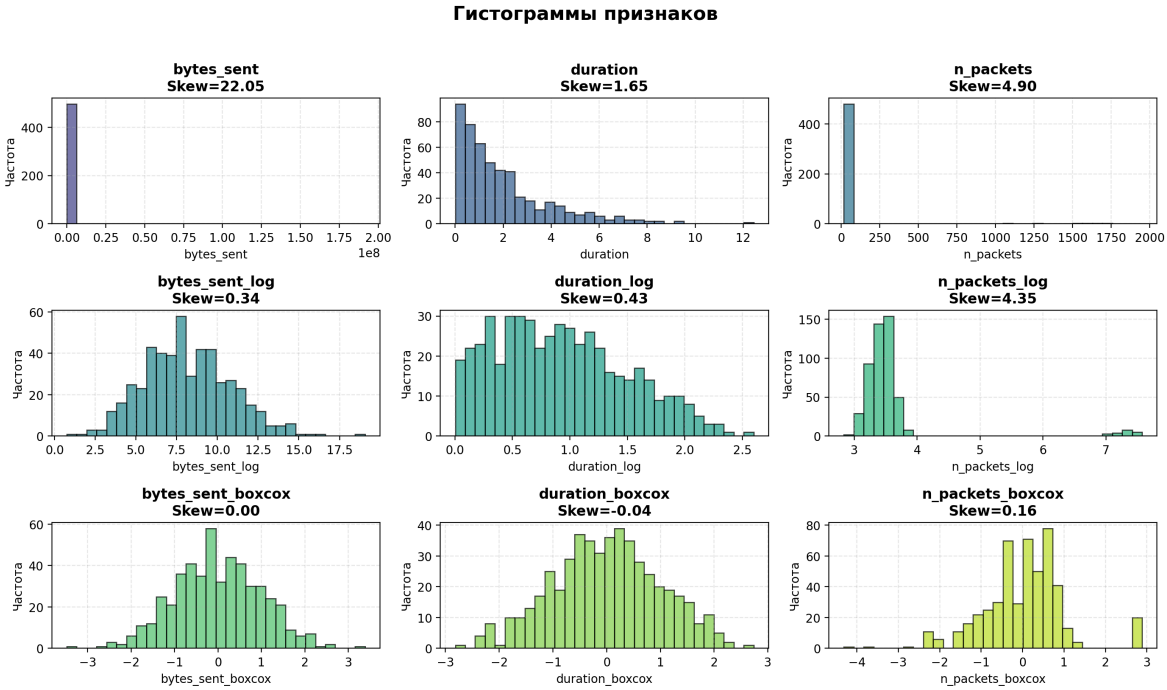
colors = plt.cm.viridis(np.linspace(0.2, 0.9, len(cols)))  # разные цвета для на

for i, col in enumerate(cols):
    ax = axes[i]
    ax.hist(df1[col], bins = 30, color = colors[i], edgecolor = 'black', alpha =

    ax.set_title(f'{col}\nSkew={skew(df1[col]):.2f}', fontsize = 12, fontweight
```

```
    ax.set_xlabel(col, fontsize = 10)
    ax.set_ylabel('Частота', fontsize = 10)
    ax.grid(True, linestyle = '--', alpha = 0.3)

plt.suptitle('Гистограммы признаков',
             fontsize = 16, fontweight = 'bold', y = 1.02)
plt.tight_layout();
```



Анализ полученных графиков.

Верхний ряд (исходные признаки): все три признака имеют экстремально скошенные распределения с длинными правыми хвостами.

Средний ряд (log): Распределения стали более симметричными, но всё ещё имеют заметную асимметрию и тяжелые хвосты.

Нижний ряд (Box-Cox): Распределения стали почти нормальными. Они симметричны, с короткими хвостами. Это именно то, что нужно для линейных моделей.

Рассмотрим пример применения преобразования Бокса-Кокса для линеаризации нелинейных зависимостей.

In [75]:
```
model_bc = LinearRegression()
model_bc.fit(df1[['bytes_sent_boxcox', 'duration_reverse', 'entropy']],
             df1['severity_score'])
pred_bc = model_bc.predict(df1[['bytes_sent_boxcox', 'duration_reverse', 'entrop
r2_bc = r2_score(df['severity_score'], pred_bc)

print("=== Сравнение моделей ===")
print()
print(f"Без log(bytes_sent): R² = {r2_raw:.4f}")
print(f"С log(bytes_sent) и reverse(duration): R² = {r2_log_reverse:.4f}")
print(f"С Box-Cox(bytes_sent) и reverse(duration): R² = {r2_bc:.4f}")
```

```
=== Сравнение моделей ===

Без log(bytes_sent): R² = 0.2761
С log(bytes_sent) и reverse(duration): R² = 0.8945
С Box-Cox(bytes_sent) и reverse(duration): R² = 0.8957
```

Преобразование Box-Cox немного увеличило значение R^2.

Поскольку преобразование Бокса-Кокса является обобщением логарифмического преобразования, то ему присущи те же ограничения.

Ограничения преобразования Бокса-Кокса.

1. Только строго положительные значения признаков.
2. Коэффициенты модели сложнее интерпретировать в исходных единицах, чем при логарифмическом преобразовании.

Предположим, что после обучения модели с логарифмированным объёмом трафика получено выражение:

$$\text{severity_score} = 0.8 + 1.5 \cdot \ln(\text{bytes_sent}).$$

Это значит: каждый раз, когда логарифм `bytes_sent` увеличиваентся на 1, `severity_scor` возрастает ровно на 1.5 пункта. А рост логарифма на 1 — это рост самого `bytes_sent` в $e^1 \approx 2.7$ раза. Поэтому получается: «если объём трафика вырос примерно в 2.7 раза, то оценка серьёзности вырастет на 1.5 пункта».

Box-Cox не просто «берёт логарифм». Он подбирает такую степень λ, чтобы распределение признака стало ближе к нормальному. Предположим для `bytes_sent` алгоритм определил $\lambda = 0.23$. Тогда преобразование выглядит так:

$$x' = \frac{\text{bytes_sent}^{0.23} - 1}{0,23}.$$

Модель, обученная на x', имеет вид:

$$\text{severity_score} = 1.7 + 1.5 \cdot x'.$$

Для этой модели интерпретация теперь звучит так: «если преобразованный признак x' вырос на 1, то `severity_score` вырастет на 1.5 пункта». Но x' — это не байты и не «во сколько раз». Это искусственная величина. Чтобы понять, что это значит в реальных байтах, нужно каждый раз делать обратный пересчёт. И самое неприятное: этот пересчёт зависит от того, сколько байт уже было.

Возьмём два случая: маленький трафик и большой. И посчитаем, на сколько байт нужно увеличить трафик, чтобы преобразованный признак вырос ровно на 1 (и тогда `severity_score` вырос бы на 1.5).

Сценарий 1: трафик небольшой — 1000 байт.

1. Считаем преобразованное значение x' для 1000 при $\lambda = 0.23$:

$$x' = \frac{1000^{0.23} - 1}{0,23} \approx 16.95.$$

2. Изменное на 1 преобразованное значение равно $x'_{new} = 17.95$.
3. Переводим это значение обратно в байты:

$$x_{new} = (\lambda \cdot x'_{new} + 1)^{1/\lambda} = (0.23 \cdot 17.95 + 1)^{1/0.23} \approx 1221.$$

4. Рост трафика: $1221 - 1000 = 221$ байт.

Вывод: чтобы поднять оценку `severity_score` на 1.5 пункта, достаточно добавить 221 байт.

Сценарий 2: трафик большой — 1 000 000 байт.

1. $x' = \frac{100000^{0.23} - 1}{0,23} \approx 89.30.$
2. $x'_{new} = 90.30.$
3. $x_{new} = (0.23 \cdot 90.30 + 1)^{1/0.23} \approx 5300000.$
4. Рост трафика: $5300000 - 1000000 = 4300000$ байт.

Вывод: чтобы поднять оценку на те же 1.5 пункта, нужно добавить 4.3 миллиона байт.

То есть нет единого правила. Нельзя сказать «если трафик вырос на X байт — поднимаем серьёзность». Нужно хранить таблицу или формулу, и каждый раз подставлять текущее значение.

Для SOC-аналитика это неудобно: ему нужно быстрое правило, а не калькулятор.

Практическое правило.

Если важна интерпретируемость → использовать логарифм.

Если важно максимальное качество → использовать Box-Cox.

Преобразование Yeo-Johnson

Одним из ограничений рассмотренных преобразований является невозможность их применения для признаков с отрицательными занчениями.

Для устранения этого ограничения применяется обобщение Box-Cox - преобразование Yeo-Johnson:

$$x^* = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda}, & \text{if } x \geq 0 \text{ и } \lambda \neq 0 \\ log(x+1), & \text{if } x \geq 0 \text{ и } \lambda = 0 \\ -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda}, & \text{if } x < 0 \text{ и } \lambda \neq 2 \\ -log(-x+1), & \text{if } x < 0 \text{ и } \lambda = 2 \end{cases}$$

Пример реализации преобразования.

In [76]:
```
from sklearn.preprocessing import PowerTransformer

# Yeo-Johnson работает с любыми числами
data_with_negatives = np.array([-5, -2, 0, 3, 10, 50])

yeo_johnson = PowerTransformer(method = 'yeo-johnson', standardize = False)
data_transformed = yeo_johnson.fit_transform(data_with_negatives.reshape(-1, 1))

print(f"Оптимальное λ: {yeo_johnson.lambdas_[0]:.4f}")
print(f"Преобразованные значения: {data_transformed.flatten()}")
```

```
Оптимальное λ: 0.4724
Преобразованные значения: [-9.45438964 -2.85166754  0.          1.95787877  4.454
16087 11.44503075]
```

Степенные преобразования

Рассмотренные преобразования являются частными случаями степенных преобразований — класс нелинейных преобразований вида $x' = x^\lambda$, где λ — это параметр, определяющий силу и направление преобразования:

- $\lambda > 1$: усиливает асимметрию (в кибербезопасности не используется);
- $\lambda = 1$: нет преобразования;
- $0 < \lambda < 1$: уменьшает асимметрию (Box-Cox);
- $\lambda \to 0$: превращается в логарифм;
- $\lambda < 0$: инвертирует данные (1/x).

Преобразование Box-Cox:

$$x' = \frac{x^\lambda - 1}{\lambda}$$

это нормализованная версия степенного преобразования x^λ.

Деление на λ и вычитание 1 нужны только для математической гладкости — чтобы при $\lambda \to 0$ формула непрерывно переходила в логарифм.

С точки зрения формы распределения, Box-Cox и x^λ дают одинаковый результат, который отличается только масштабом.

Логарифм — это предельный случай степенного преобразования при $\lambda \to 0$.

Квадратный корень — это степенное преобразование при $\lambda = 0.5$.

Литература

1. Криволапов, С. Я. Анализ данных. Методы теории вероятностей и математической статистики на языке Python : учебное пособие / С.Я. Криволапов. – Москва : ИНФРА-М, 2025. – 678 с.

2. Груздев А.В. Предварительная подготовка данных в Python. Том 2: План, примеры и метрики качества. – М.: ДМК Пресс, 2023. – 814 с.
3. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2022 — Часть 1 — 2022. — 107 с.
4. Демидова, Л. А. Разведочный анализ данных. Python : учебно-методическое пособие / Л. А. Демидова. — Москва : РТУ МИРЭА, 2023 — Часть 2 — 2023. — 92 с.